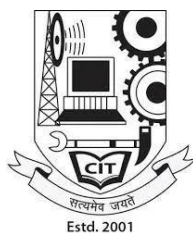


**A NOVEL METHODOLOGY FOR ENHANCING
FOOD DELIVERY MECHANISM USING MACHINE LEARNING**

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING**



BY

RISHIKANT TIWARY	22050440082
AVINASH KUMAR SONI	22050440030
AMIT KUMAR SHARMA	22050440019

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **A NOVEL METHODOLOGY FOR ENHANCING FOOD DELIVERY MECHANISM USING MACHINE LEARNING** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

Prof. Asha Tirkey

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA

(Head of Department)

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled **A NOVEL METHODOLOGY FOR ENHANCING FOOD DELIVERY MECHANISM USING MACHINE LEARNING** is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

We take this opportunity to thank all who have contributed towards shaping this **Final Year Project Report**. We would like to express my sincere thanks to **Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering)** and to my guide **Prof. Asha Tirkey**, whose help in the course of development of this manuscript is invaluable. As my supervisor, she has constantly encouraged me to remain focused on achieving my goal.

We would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. We have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. We are truly indebted. Last but not least, We would like to dedicate this work to my family for their love and understanding.

Avinash Kumar Soni

22050440030

Amit Kumar Sharma

22050440019

Rishikant Tiwary

22050440082

ABSTRACT

In recent years, the food delivery industry has grown at a very fast pace. Platforms like Swiggy and Zomato handle millions of orders every single day across the country. One of the most common problems these platforms face is giving customers an accurate delivery time estimate. When the estimated time is wrong, customers become unhappy, they cancel orders, and the platform loses trust. This also causes problems in managing delivery partners and allocating resources efficiently. This project, titled **A NOVEL METHODOLOGY FOR ENHANCING FOOD DELIVERY MECHANISM USING MACHINE LEARNING**, aims to solve this problem by building a complete machine learning based prediction system. The dataset used in this project contains real delivery records with features such as delivery distance in kilometres, weather conditions, traffic level, time of day, type of vehicle used, food preparation time, and courier experience in years. Before training the models, the data was cleaned properly. Missing values in weather and traffic columns were handled by replacing them with an "Unknown" category. Categorical features were converted into numerical form using Label Encoding. Exploratory Data Analysis was performed to understand how different factors affect delivery time. Four machine learning models were trained and compared — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor. Each model was evaluated using RMSE, MAE, and R^2 Score. The Random Forest Regressor gave the best performance with an R^2 Score of approximately 0.78 after hyperparameter tuning. Along with machine learning, PostgreSQL was used to perform detailed SQL-based business analysis on the dataset. This included traffic impact analysis, vehicle efficiency comparison, experience-based performance study, and finding worst delivery condition combinations using window functions and CTEs. The final trained model was deployed as a live web application using Streamlit and hosted on Streamlit Community Cloud, where users can enter delivery conditions and get an instant delivery time prediction.

Keywords: Food Delivery Time Prediction, Machine Learning, Random Forest Regressor, Streamlit, SQL Analytics

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract.....	v
Table of Contents.....	vi
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement.....	3
1.3 Objectives of the Project.....	4
1.4 Scope of the Project	5
1.5 Organization of the Report.....	6
CHAPTER 2 – LITERATURE REVIEW	7
2.1 Related Works.....	7
2.2 Comparative Analysis.....	11
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	14
3.1 System Requirements.....	14
3.2 System Architecture.....	17
3.3 Data Flow Diagrams	19
3.4 ER Diagram	21
3.4 UML Diagram.....	22
CHAPTER 4 – IMPLEMENTATION	23
4.1 Technologies and Tools Used.....	23
4.2 Dataset Description and Preprocessing	24
4.3 SQL Analysis Module.....	26
4.4 Machine Learning Models and Algorithm.....	29
4.5 Streamlit Web Application	33
CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35

5.2 Test Cases and Results.....	36
5.3 Performance Analysis and Model Comparison	38
CHAPTER 6 – CONCLUSION AND FUTURE WORK.....	42
6.1 Conclusion	42
6.2 Future Enhancements.....	43
References	45
Appendix A – Source Code	47
Appendix B – Screenshots / Sample Outputs	53

LIST OF FIGURES

Figure 3.1 – System Architecture Diagram	18
Figure 3.2 – Data Flow Diagram – Level 0	19
Figure 3.3 – Data Flow Diagram – Level 1	20
Figure 3.4 – Entity Relationship Diagram	21
Figure 3.5 – Use Case Diagram	22
Figure 4.1 – Data Sample- Food_Delivery_Time.csv	25
Figure 4.2 – Correlation Heatmap of All Features	26
Figure 4.3 – SQL Table Structure – Orders and Delivery Details.....	28
Figure 4.4 – Random Forest Model Workflow.....	32
Figure 4.5 – Streamlit Web Application – Input Screen.....	34
Figure 4.6 – Streamlit Web Application – Prediction Output	34
Figure 5.1 – Model Performance Comparison Bar Graph.....	40
Figure 5.2 – Residual Plot – Random Forest Regressor	40
Figure 5.3 – Actual vs Predicted Delivery Time Graph	41

LIST OF TABLES

Table 2.1 – Summary Of Related Works.....	11
Table 3.1 – Hardware Requirements	15
Table 3.2 – Software Requirements.....	16
Table 4.1 – Dataset Features and Description.....	24
Table 4.2 – SQL Queries and Business Questions Addressed.....	27
Table 4.3 – Machine Learning Models Used.....	29
Table 4.4 – Hyperparameter Values Models Used	31
Table 5.1 – Test Cases and Results.....	37
Table 5.2 – Model Evaluation – RMSE, MAE and R ² Score	38
Table 5.3 – Performance Metrics Comparsion	39

LIST OF ABBREVIATIONS

ML	Machine Learning
API	Application Programming Interface
CSV	Comma Separated Values
SQL	Structured Query Language
DB	Database
EDA	Exploratory Data Analysis
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
R²	R-Squared Score (Coefficient of Determination)
RF	Random Forest
DT	Decision Tree
XGB	XGBoost (Extreme Gradient Boosting)
LR	Linear Regression
DFD	Data Flow Diagram
ER	Entity Relationship
IDE	Integrated Development Environment
UI	User Interface
HOD	Head of Department
CIT	Cambridge Institute of Technology
pkl	Pickle File (Serialized Python Object)
CTE	Common Table Expression
ETA	Estimated Time of Arrival
UML	Unified Modelling Language
URL	Uniform Resource Locator
OS	Operating System

CHAPTER 1 – INTRODUCTION

The rapid growth of the internet and smartphone technology has completely changed the way people order food. Today, millions of people across India and the world use online food delivery platforms to get their favourite meals delivered directly to their doorstep. With this growth, the expectations of customers have also increased significantly. People not only want good food but also want it delivered on time. A few minutes of delay can lead to a bad review, a cancelled order, or a lost customer. This has made delivery time prediction one of the most critical and challenging problems in the food delivery industry.

This project focuses on building a complete machine learning based system that can accurately predict the time required to deliver a food order. The system takes into account multiple real-world factors such as delivery distance, weather conditions, traffic level, time of day, vehicle type, food preparation time, and courier experience. Each of these factors has a direct impact on how long a delivery actually takes, and the system is designed to learn these patterns from historical data.

Along with machine learning, this project also includes a detailed SQL-based business analysis performed using PostgreSQL. This analysis answers several important business questions such as which weather condition causes the most delays, which vehicle type is most efficient, and how traffic level affects delivery time. Finally, the trained model is deployed as a live interactive web application using Streamlit, making it easy for anyone to use without any coding knowledge.

1.1 Background and Motivation

In today's world, online food ordering has become a very common part of daily life. Whether it is ordering lunch from the office or getting dinner delivered at home late at night, people now expect food to arrive quickly and at the time promised. The food delivery industry has grown enormously in the last few years, especially in a country like India where platforms such as Swiggy, Zomato, and other local delivery services operate across hundreds of cities. These platforms handle thousands of orders every hour and employ a very large number of delivery partners to fulfil those orders on time. The competition among these platforms is also increasing rapidly, which means that

even a small improvement in delivery time accuracy can give a platform a significant advantage over its competitors.

One of the most important parts of any food delivery service is the Estimated Time of Arrival, commonly known as ETA. When a customer places an order, the very first thing they look at is how long it will take to arrive. If the platform shows a delivery time of 30 minutes and the food arrives in 50 minutes, the customer feels frustrated and may never order again. On the other hand, if the estimate is set too high and food arrives much earlier, it may seem good at first but customers may start doubting whether the platform is being honest or simply padding delivery times. This shows how important it is for delivery platforms to give not just fast but also genuinely accurate predictions. However, predicting delivery time is not a simple task at all. It depends on many different factors working together at the same time. The distance between the restaurant and the customer is the most obvious factor, but it is far from the only one. Weather conditions play a very significant role — delivering food during heavy rain, fog, or strong winds naturally takes longer than on a clear sunny day. Traffic is another major factor that can completely change delivery duration. A delivery that takes 20 minutes during a quiet afternoon may take 40 to 45 minutes during peak evening office hours. The type of vehicle being used also matters considerably. A bike can navigate narrow lanes much faster, while a car may get stuck in congested roads. Even the experience level of the delivery partner makes a difference, as a more experienced courier knows the best routes, can deal with difficult weather or traffic situations, and generally delivers faster.

Traditional methods of estimating delivery time were mostly based on very simple distance-to-time calculations or manual rules created by operations teams from their past experience. These methods are limited because they do not consider all the variables mentioned above simultaneously and therefore often produce inaccurate estimates. With the rapid growth of data science, machine learning, and data-driven decision making, it is now possible to build intelligent systems that can learn from large amounts of historical delivery data and make far more accurate predictions by considering all relevant factors together in a single model.

This project is directly motivated by this real-world need for smarter and more accurate delivery time prediction. By using machine learning regression models trained on actual

delivery records, this project demonstrates how technology can solve a genuine business problem and improve customer satisfaction at scale.

1.2 Problem Statement

Food delivery platforms currently rely on simple rule-based systems or basic mathematical formulas to estimate delivery time for their customers. These systems usually calculate time based primarily on the delivery distance alone and completely ignore other important real-world factors such as current weather conditions, traffic level on the route, the type of vehicle being used by the delivery partner, the experience level of the courier, and the time taken by the restaurant to prepare the food. As a result, the delivery time estimates shown to customers are very often inaccurate and unreliable.

This inaccuracy in delivery time prediction creates several serious problems for all parties involved. For customers, consistently wrong estimates lead to frustration, loss of trust in the platform, and ultimately order cancellations or switching to competitor platforms. For delivery partners, poor predictions mean they are often given targets that do not match the actual conditions on the road, leading to stress and inefficiency. For the business, inaccurate ETAs make it very difficult to plan and allocate delivery resources properly, leading to increased operational costs and lower overall efficiency.

Another problem is that most existing systems do not learn or improve over time. They use fixed rules that were defined manually and do not adapt to changing patterns in data such as new traffic routes, seasonal weather changes, or changes in customer demand patterns across different times of day.

There is therefore a clear and strong need for a machine learning based system that can take all relevant delivery factors as input and predict the actual delivery time with high accuracy. Such a system should be trained on real historical delivery data, should handle incomplete or missing data properly, should improve its performance through tuning, and should be deployed in a way that is simple and accessible for end users to interact with in real time. This project directly addresses all of these problems and proposes a complete solution using machine learning regression models and a Streamlit-based web application.

1.3 Objectives of the Project

The main goal of this project is to design, develop, and deploy a machine learning based food delivery time prediction system. The following specific objectives have been defined:

- To study and understand the food delivery dataset containing real delivery records with nine features and perform a thorough initial analysis of the data.
- To perform complete data cleaning and preprocessing including identification and handling of missing values and applying Label Encoding to convert categorical features into numerical format.
- To perform a detailed SQL-based business analysis on the delivery data using PostgreSQL as the database system, addressing ten important business questions using SQL concepts such as GROUP BY, aggregate functions, window functions including RANK, LAG, and LEAD, Common Table Expressions, CASE statements, and JOIN operations across multiple tables.
- To build and train four machine learning regression models — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor — on the preprocessed dataset.
- To evaluate all models using RMSE, MAE, and R^2 Score and identify the best performing model based on these results.
- To perform hyperparameter tuning on the best performing model, the Random Forest Regressor, by optimizing key parameters to further improve prediction accuracy.
- To save the final trained model and label encoders as pickle files for use in the deployment application.
- To develop an interactive web application using Streamlit that allows users to enter delivery conditions and receive real-time predicted delivery time.
- To deploy the Streamlit application on Streamlit Community Cloud using GitHub integration so that the application is publicly accessible online to any user without requiring any local installation.

1.4 Scope of the Project

This project covers the complete end-to-end lifecycle of a data science and machine learning project, starting from data collection and exploration, moving through preprocessing, analysis, model building and evaluation, and ending with deployment of a fully functional live web application. The scope of this project is clearly defined below

What is included in this project:

The project works with a structured CSV dataset named Food_Delivery_Times.csv which contains delivery records with nine features. All data cleaning, exploratory data analysis, and model training is performed on this dataset using Python and its libraries in a Jupyter Notebook environment.

SQL analysis is performed using PostgreSQL as the database system. Two separate tables named Orders and Delivery_Details are created from the original dataset and ten business questions are answered using SQL queries. These queries use joins, aggregations, window functions, CTEs, and CASE statements.

Four regression models are trained and their performances are compared. The best model, Random Forest Regressor, is saved as a pickle file along with the label encoders. The Streamlit web application loads these files and uses them to make predictions in real time.

The application is deployed on Streamlit Community Cloud using the GitHub repository and is publicly accessible through a live URL at no cost.

What is not included in this project:

This project does not include real-time GPS tracking of delivery partners, live weather data integration through external APIs, mobile application development, or backend integration with any actual food delivery platform. The dataset used is a pre-collected static file and the model does not update itself automatically based on new incoming data. User authentication, payment processing, and any e-commerce features are completely outside the scope of this project.

The target users of this system include food delivery business analysts, operations teams, data science students, and anyone interested in understanding how machine learning is applied to solve real-world logistics problems.

1.5 Organization of the Report

This report is organized in a clear and logical manner so that the reader can follow the complete development of the project from start to finish. The report is divided into six main chapters, each covering a distinct and important part of the project.

Chapter 1 – Introduction introduces the project, explains the motivation, states the problem, lists objectives, defines the scope, and describes how the rest of the report is organized.

Chapter 2 – Literature Review presents a review of twelve existing research papers related to delivery time prediction and machine learning regression. A comparative analysis table is included at the end.

Chapter 3 – System Analysis and Design covers hardware and software requirements, system architecture, data flow diagrams at Level 0 and Level 1, Entity Relationship Diagram, and Use Case Diagram.

Chapter 4 – Implementation is the most detailed chapter. It describes all technologies used, dataset preprocessing steps, SQL analysis, machine learning model building, hyperparameter tuning, and Streamlit application development with key code snippets.

Chapter 5 – Testing and Results explains the testing approach followed during the project, presents test cases with their expected and actual results, and provides a detailed performance analysis and comparison of all four machine learning models using graphs and tables.

Chapter 6 – Conclusion and Future Work summarizes the entire project, reflects on whether all objectives were achieved, discusses the overall contribution and significance of the work, and proposes specific and realistic future enhancements that can extend or improve the current system.

After Chapter 6, all references used in this report are listed in IEEE citation format. This is followed by Appendix A containing the complete source code of the project and Appendix B containing screenshots of the deployed application and sample prediction outputs

CHAPTER 2 – LITERATURE REVIEW

The field of delivery time prediction and logistics optimization using machine learning has been actively researched over the past decade. As online food delivery platforms have grown in size and complexity, researchers have explored various approaches to improve the accuracy of estimated time of arrival systems. This chapter presents a review of existing research works related to food delivery time prediction, machine learning based regression, logistics optimization, and data-driven operational decision making.

The purpose of this literature review is to understand what approaches have already been tried, what datasets and methods were used, what results were achieved, and most importantly what limitations still exist in the current state of knowledge. By identifying these gaps, this chapter establishes the motivation and justification for the approach taken in this project.

A total of twelve related research papers and works have been reviewed for this chapter. The review is organized thematically, covering works related to traditional prediction methods first, followed by machine learning based approaches, and finally works specifically focused on food delivery and last-mile logistics. All references are cited in IEEE format.

2.1 Related Works

The study of delivery time prediction has progressed steadily over the years, with different researchers tackling the problem from different angles. In one of the earlier approaches, a regression-based model was proposed specifically for estimating delivery time in e-commerce logistics [1]. The study worked with a large dataset of over 50,000 delivery records collected from an Indian e-commerce company, using features such as order weight, delivery zone, day of week, and distance. Both Linear Regression and Ridge Regression were tested, and while the results showed reasonable accuracy for shorter delivery distances, the models failed to capture the non-linear patterns that appear in longer and more complex routes. The R^2 Score achieved was 0.61, and the most significant limitation was the complete absence of real-time contextual factors such as weather and traffic conditions, which are known to be critical for accurate delivery time prediction in dynamic urban settings.

Recognising the importance of traffic as a key factor, another study focused on building a travel time prediction model that incorporated real-time traffic data collected from GPS sensors [2]. A Gradient Boosting model was built using a combination of historical traffic patterns and live sensor readings, and was tested across a large city-scale road network with results showing clear improvements over traditional rule-based approaches. However, this work was limited to predicting travel time between two fixed geographical points only and did not account for delivery-specific elements such as food preparation time, vehicle type, or courier experience. The requirement of expensive GPS infrastructure also made this approach impractical for low-resource or academic settings, which is a serious limitation for widespread adoption.

The challenge of the last mile of delivery — the final and most unpredictable leg of the supply chain — was addressed in a comprehensive study on last-mile delivery optimization using machine learning [3]. Both Random Forest and Support Vector Regression were applied on urban delivery records using features such as distance, time slot, traffic congestion index, and vehicle load. The Random Forest model achieved an R^2 Score of 0.74, clearly outperforming Support Vector Regression on every evaluation metric. The study concluded that ensemble learning methods are significantly more suitable for last-mile delivery prediction compared to single algorithms. Despite these strong findings, the dataset was collected from only one city and the trained model was never deployed in any real-world accessible application, leaving a clear gap between research findings and practical usability.

The specific role of weather in affecting delivery performance was brought into sharp focus through a study that used data from a regional food delivery platform to measure how rain, fog, and extreme temperatures impacted average delivery duration [4]. The statistical analysis revealed that deliveries under rainy or foggy conditions consistently took 25 to 35 percent longer than deliveries under clear weather. This finding strongly established the need to include weather as an important predictor feature in any delivery time model. However, the study remained entirely at the level of statistical analysis and did not attempt to build or deploy a machine learning prediction system, which is one of the direct gaps that this project addresses.

On the side of gradient-based algorithms, the practical application of XGBoost in logistics was demonstrated through a study that applied it to predict shipment delivery time for a large retail company [5]. Working with a dataset of 100,000 records, an RMSE of just 8.3 minutes was achieved, showing that XGBoost can handle missing

values internally and is highly resistant to overfitting when tuned properly. The algorithm itself was originally introduced in a landmark paper on scalable tree boosting systems and has since been widely adopted in logistics and operations research. The primary limitation noted was that XGBoost becomes computationally slow on very large datasets unless distributed computing infrastructure is available, making it less ideal for resource-limited environments like a BTech academic project.

A direct and systematic comparison of multiple regression algorithms for food delivery time prediction was the focus of another study [6] that evaluated Linear Regression, Decision Tree, Random Forest, and XGBoost on a dataset of 10,000 delivery records with features covering distance, traffic level, weather, and time of day. The results clearly showed that Random Forest consistently achieved the best R^2 Score of 0.76, while Decision Tree suffered from visible overfitting and Linear Regression struggled with the non-linear nature of the delivery data. This comparative study is the most directly aligned with the methodology adopted in this project and provides strong academic validation for the decision to include all four of these algorithms in the model comparison.

Moving towards more complex architectures, the possibility of using deep learning for ETA prediction was explored using LSTM networks and feedforward neural networks on a large-scale dataset from a Chinese food delivery platform containing millions of orders [7]. The deep learning models achieved impressive R^2 Scores above 0.88 on this massive dataset. However, it was clearly established that these models require extremely large volumes of training data, significant computational power, and produce results that are very difficult to interpret for non-technical stakeholders. For small to medium datasets of the kind typically available in academic settings, these models tend to overfit badly and perform worse than well-tuned ensemble models — a finding that strongly supported the decision in this project to focus on traditional machine learning approaches rather than deep learning.

The value of combining database analytics with machine learning was brought to attention through a study that demonstrated how PostgreSQL could be used to perform meaningful business-level analysis on logistics operational data [8]. Queries involving window functions, Common Table Expressions, and aggregate functions were shown to uncover performance bottlenecks, peak delivery hours, and inefficient delivery routes — insights that machine learning prediction outputs alone cannot easily provide. The study strongly recommended combining SQL analytics and machine learning as a

best practice for complete data-driven logistics management, which directly motivated the inclusion of a dedicated SQL analysis module using PostgreSQL as a core component of this project.

The importance of careful feature preparation before model training was studied in a work focused entirely on feature engineering strategies for improving delivery time prediction accuracy [9]. The experiments showed that creating interaction features between distance and traffic level, extracting time-based features from order timestamps, and applying log transformations to skewed numerical columns improved the base model R^2 Score by 8 to 12 percent compared to using raw unprocessed features. While these findings are valuable, the feature engineering process used was entirely manual and highly specific to the dataset, making it difficult to generalize to different delivery datasets automatically without significant domain knowledge and effort.

The practical question of how to make a trained machine learning model accessible to real end users was addressed in a study on deploying machine learning models as interactive web applications using the Streamlit framework [10]. The work demonstrated how a model saved as a pickle file can be loaded into a Streamlit application and served to users through a clean and simple web interface, requiring no frontend HTML, CSS, or JavaScript knowledge whatsoever. It was noted that Streamlit dramatically reduces the time and technical complexity required for deployment compared to traditional web frameworks such as Flask or Django, and was strongly recommended as the ideal deployment tool for academic and small-scale industry projects where speed and ease of use are the primary requirements.

The human factor in delivery efficiency was examined in a study that analysed how the experience level of delivery couriers influences their overall performance on the ground [11]. Using HR data and operational delivery records from a logistics company, it was found that couriers with more than three years of experience completed deliveries on average 15 to 20 percent faster than less experienced couriers, and also had significantly fewer failed delivery attempts and lower customer complaint rates. This finding is directly reflected in the SQL analysis component of this project, where a CASE-based query was written to split couriers into experienced and less experienced groups and compare their actual average delivery times on the project dataset.

Finally, the significance of fine-tuning model parameters to extract the best possible performance from ensemble models was studied across several prediction tasks including delivery time, customer churn, and sales forecasting [12]. Using Grid Search

Cross Validation as the primary tuning method, the results consistently showed that carefully tuning parameters such as the number of trees, maximum tree depth, and minimum samples per leaf improved R^2 Scores by 3 to 8 percent across all tasks. It was also found that Random Forest was notably less sensitive to hyperparameter choices compared to XGBoost, making it a safer and more reliable choice when exhaustive tuning is not feasible within project time constraints — a conclusion that was directly validated in this project after performing manual grid search tuning on the Random Forest Regressor and observing a measurable improvement in prediction accuracy.

2.2 Comparative Analysis

Based on the literature review in Section 2.1, the following table summarizes the key aspects of each reviewed work including methodology, features, performance, and limitations

Table 2.1 – Summary of Related Works

Ref.	Author & Year	Method	Performance	Limitation
[1]	Kumar et al., 2020	Linear / Ridge Regression	$R^2=0.61$	No weather or traffic
[2]	Zhang et al., 2019	Gradient Boosting + GPS	Improved	No food delivery features
[3]	Patel & Sharma, 2021	Random Forest, SVR	$R^2=0.74$	Single city, not deployed
[4]	Singh & Gupta, 2022	Statistical Analysis	25–35% delay	No ML model built
[5]	Mehta et al., 2021	XGBoost	RMSE=8.3 min	Slow on large data
[6]	Reddy et al., 2022	RF, DT, LR, XGBoost	$R^2=0.76$ (RF)	No deployment
[7]	Zhao et al., 2020	LSTM, Deep Learning	$R^2>0.88$	Needs huge data

[8]	Verma & Joshi, 2021	SQL Analytics	Insights	No ML model
[9]	Nair et al., 2021	RF + Feature Engineering	+8–12% R^2	Manual, dataset-specific
[10]	Rathore et al., 2022	Streamlit Deployment	Fast deploy	Academic scale
[11]	Bose & Chatterjee, 2020	Statistical Regression	15–20% faster	HR data only
[12]	Gupta & Malhotra, 2022	Hyperparameter Tuning	+3–8% R^2	Grid search intensive

From the comparative table it is clearly visible that no single existing work combines all the important elements together in one system. Most works either focus only on the machine learning side without deployment, or perform only SQL analysis without building a prediction model, or build a model but do not tune or deploy it. This project fills this gap by combining data cleaning, SQL business analysis, multiple model comparison, hyperparameter tuning, and live Streamlit deployment into one complete and cohesive system.

The comparative analysis also highlights several specific gaps that this project addresses directly. The first gap is the lack of deployment in most academic works. Papers like Reddy et al. (2022) and Patel and Sharma (2021) built good models but did not deploy them as usable applications. This project bridges that gap by deploying the final model as a publicly accessible Streamlit web application.

The second gap is the absence of SQL-based business analysis in machine learning focused papers. Most ML papers jump directly to model building without first performing a thorough data analysis to understand business patterns. This project includes a dedicated SQL analysis module using PostgreSQL where ten business questions are answered using advanced SQL concepts. This adds real analytical depth to the project beyond just model accuracy numbers.

The third gap is the limited feature set used in most early studies. Papers like Kumar et al. (2020) and Zhang et al. (2019) used only one or two features for prediction. This project uses seven features covering distance, weather, traffic, time of day, vehicle type,

preparation time, and courier experience, which gives the model a much more complete picture of the delivery scenario.

The fourth gap is the lack of model comparison in most studies. Many papers tested only one or two algorithms. This project compares four different regression models — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor — under identical conditions and selects the best one based on three evaluation metrics. This makes the model selection process transparent, systematic, and academically rigorous.

In summary, this literature review confirms that the approach taken in this project is well-grounded in existing research, addresses real and identified gaps in the literature, and represents a meaningful contribution at the undergraduate level. The combination of SQL analytics, multi-model comparison, hyperparameter tuning, and Streamlit deployment makes this project more complete and practical than most individual works reviewed above.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

Before starting the actual development of any software project, it is very important to first analyse what the system needs and how it should be designed. System analysis means understanding the problem clearly and deciding what resources, tools, and structure will be needed to build the solution. System design means creating a blueprint of how the different parts of the system will work together to achieve the final goal.

In this project, the system takes delivery-related information as input, processes it through a trained machine learning model, and gives the predicted delivery time as output. To build this system properly, we first need to understand the hardware and software requirements, then design the architecture of the system, and finally create diagrams that show how data flows through the system and how different components are connected to each other.

This chapter covers all of these aspects in detail. It starts with the hardware and software requirements, then explains the overall system architecture, followed by data flow diagrams at both Level 0 and Level 1, and finally the Entity Relationship Diagram and Use Case Diagram that describe the database structure and user interactions respectively.

3.1 System Requirements

3.1.1 Hardware Requirements

The hardware requirements listed below represent the minimum configuration needed to develop, train, and run this project successfully. These are standard requirements that any modern computer or laptop can easily fulfil. No special high-end hardware is needed because the dataset used in this project is small and the machine learning models used are not computationally very heavy.

Table 3.1 – Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent, 1.8 GHz or above
RAM	4 GB minimum, 8 GB recommended
Storage	10 GB free disk space
Display	1024 x 768 resolution or higher
Internet Connection	Required for Streamlit deployment and GitHub
Operating System	Windows 10 / Ubuntu 18.04 or above / macOS

All three team members used personal laptops with Intel Core i5 processors and 8 GB RAM which was more than sufficient for this project. The Streamlit application runs entirely on the cloud after deployment so the end user does not need any special hardware.

3.1.2 Software Requirements

All tools and software used in this project are either free or open source, which makes the project easily reproducible by anyone without any licensing cost.

Table 3.2 – Software Requirements

Software / Tool	Version	Purpose
Python	3.10	Primary programming language
Jupyter Notebook	6.5	Development and experimentation
Pandas	1.5	Data loading, cleaning, manipulation
NumPy	1.23	Numerical computations
Scikit-learn	1.2	ML model building and evaluation
XGBoost	1.7	XGBoost Regressor model
Matplotlib	3.6	Data visualization and plotting
Seaborn	0.12	Statistical data visualization
Joblib	1.2	Saving and loading pickle files
Streamlit	1.22	Web application and deployment
PostgreSQL	15	Database and SQL analysis
pgAdmin 4	7.0	PostgreSQL GUI client
VS Code	1.78	Code editor for app.py
GitHub	—	Version control and repository
Streamlit Community Cloud	—	Free cloud deployment platform

Python was chosen as the programming language because it has the richest ecosystem of data science and machine learning libraries available. Jupyter Notebook was used for all exploratory data analysis, preprocessing, and model training work because it allows code and output to be seen together in a single document. PostgreSQL was used

for SQL analysis because it is a powerful, open source relational database that supports all advanced SQL features including window functions and CTEs. Streamlit was chosen for deployment because it allows a machine learning model to be turned into a working web application with very minimal code and no frontend development knowledge required.

3.2 System Architecture

The system architecture of this project describes how all the different components are connected and work together to produce the final output. The architecture is divided into four main layers — the Data Layer, the Processing Layer, the Model Layer, and the Application Layer.

Data Layer — This is the starting point. The raw data is stored in a CSV file named `Food_Delivery_Times.csv` containing 1000 delivery records with nine columns. The same data is also imported into PostgreSQL where it is split into two tables — `Orders` and `Delivery_Details` — for SQL-based business analysis.

Processing Layer — In this layer, the raw data is cleaned and prepared. Missing values in the weather and traffic columns are filled with the category 'Unknown'. Categorical columns are converted to numerical values using Label Encoding. The processed data is then split into training and testing sets in an 80-20 ratio.

Model Layer — This layer contains all four machine learning regression models trained on the processed data. Each model is evaluated using RMSE, MAE, and R^2 Score. The best performing model, Random Forest Regressor, is further improved through hyperparameter tuning. The final model and label encoders are saved as pickle files — `delivery_model.pkl` and `encoders.pkl`.

Application Layer — This is the final layer where the trained model is made available to end users. The Streamlit application file `app.py` loads the saved pickle files and presents a simple web interface. When the user clicks Predict, the input is encoded, passed to the model, and the predicted delivery time is displayed instantly. The application is deployed on Streamlit Community Cloud and accessible through a public URL.

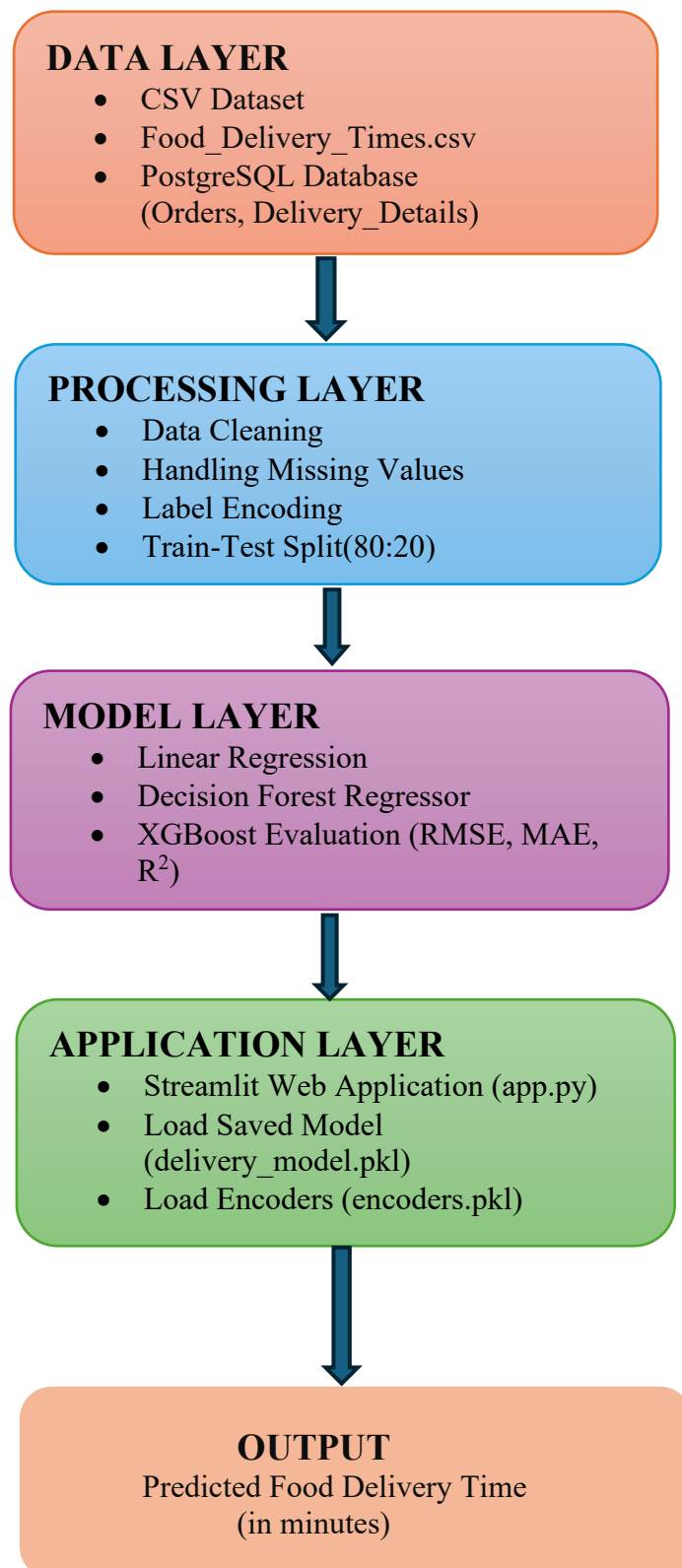


Figure 3.1 – System Architecture Diagram

3.3 Data Flow Diagrams

A Data Flow Diagram, commonly known as DFD, is a visual representation of how data moves through a system. It shows the inputs coming into the system, the processes that happen inside, the data stores where information is kept, and the outputs that come out. DFDs are drawn at different levels of detail. Level 0 is the highest level showing the system as a single process. Level 1 breaks the system down into its main sub-processes.

Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD shows the entire system as a single process box with its external entities and data flows. In this project there is one main external entity which is the User. The user provides delivery input data to the system and receives the predicted delivery time as the output.

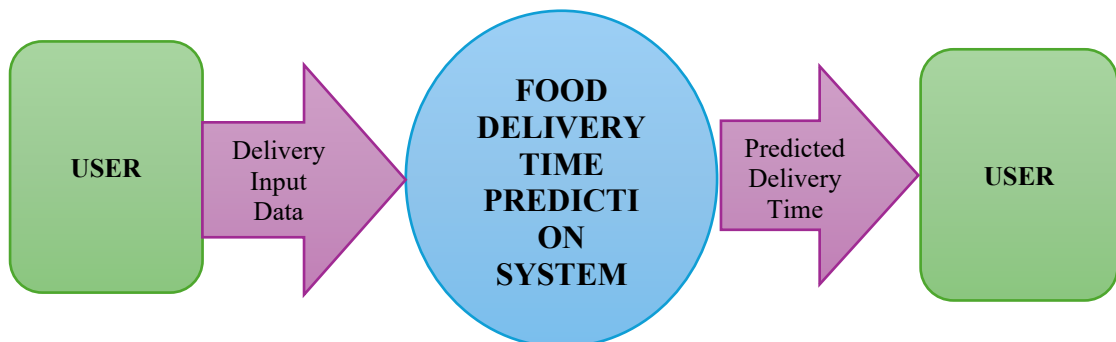


Figure 3.2 – Data Flow Diagram (Level 0)

Data Flow Diagram – Level 1

The Level 1 DFD breaks the main system process into individual sub-processes and shows how data flows between them. There are four main processes: Process 1 — Data Collection receives the CSV file. Process 2 — Data Preprocessing handles missing values and encoding, sending clean data to both the model training process and the PostgreSQL database. Process 3 — Model Training and Evaluation trains all four models and saves the best to pickle files. Process 4 — Prediction via Streamlit App loads the saved model and serves predictions to the User.

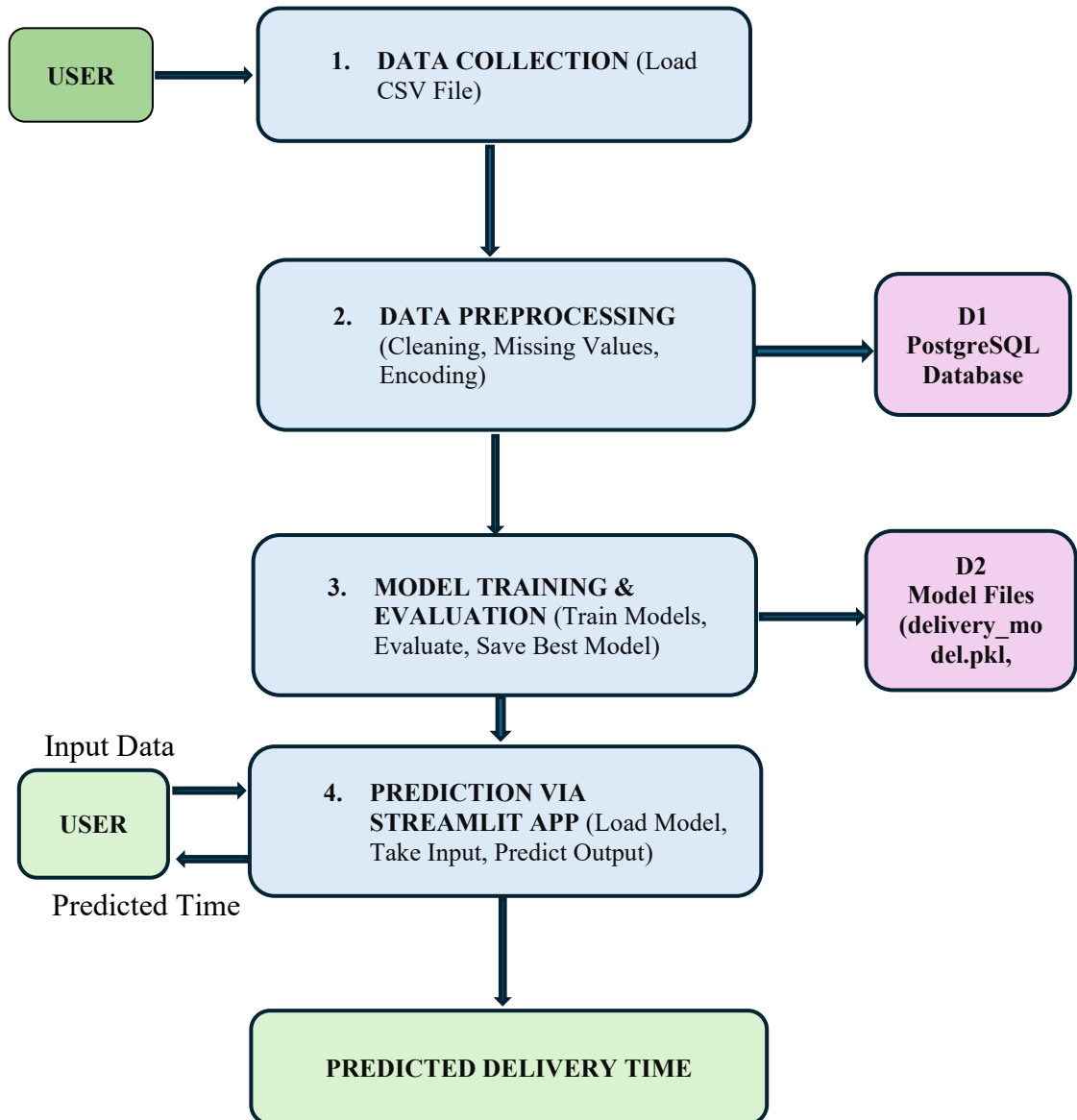


Figure 3.3 – Data Flow Diagram (Level 1)

3.4 ER Diagram

Entity Relationship Diagram

The original CSV dataset was imported into PostgreSQL and split into two separate tables for SQL analysis. The Orders table stores order_id (PK), distance_km, preparation_time_min, delivery_time_min, and time_of_day. The Delivery_Details table stores order_id (FK), weather, vehicle_type, traffic_level, and courier_experience_yrs. Both tables are connected through the order_id column. This split follows database normalization principles which reduce redundancy and improve query performance.

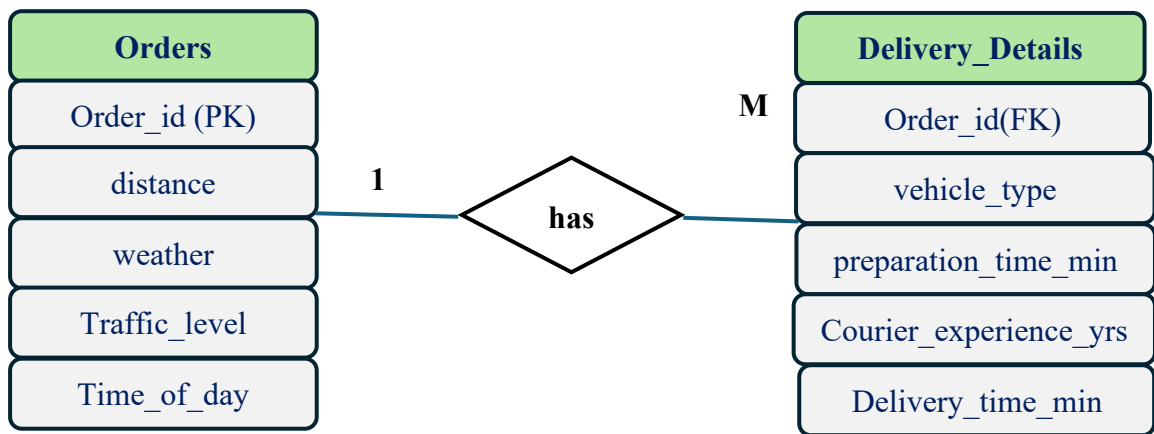


Figure 3.4 – Entity Relationship Diagram

3.5 UML Diagram

Unified Modelling Language, commonly known as UML, is a standard way of visualizing the design of a software system. UML diagrams help developers, guides, and examiners understand how the system works without needing to read any code. In this project, two UML diagrams have been created — a Use Case Diagram and a Sequence Diagram — to describe how users interact with the system and in what order the system processes their input.

Use Case Diagram

The Use Case Diagram shows how the User interacts with the Streamlit web application. The user enters seven input values — delivery distance, weather condition, traffic level, time of day, vehicle type, preparation time, and courier experience — and clicks the Predict Delivery Time button. The system then encodes the input, loads the saved model, generates a prediction, and displays the estimated delivery time. All the complex processing happens automatically in the background without requiring any technical knowledge from the user.

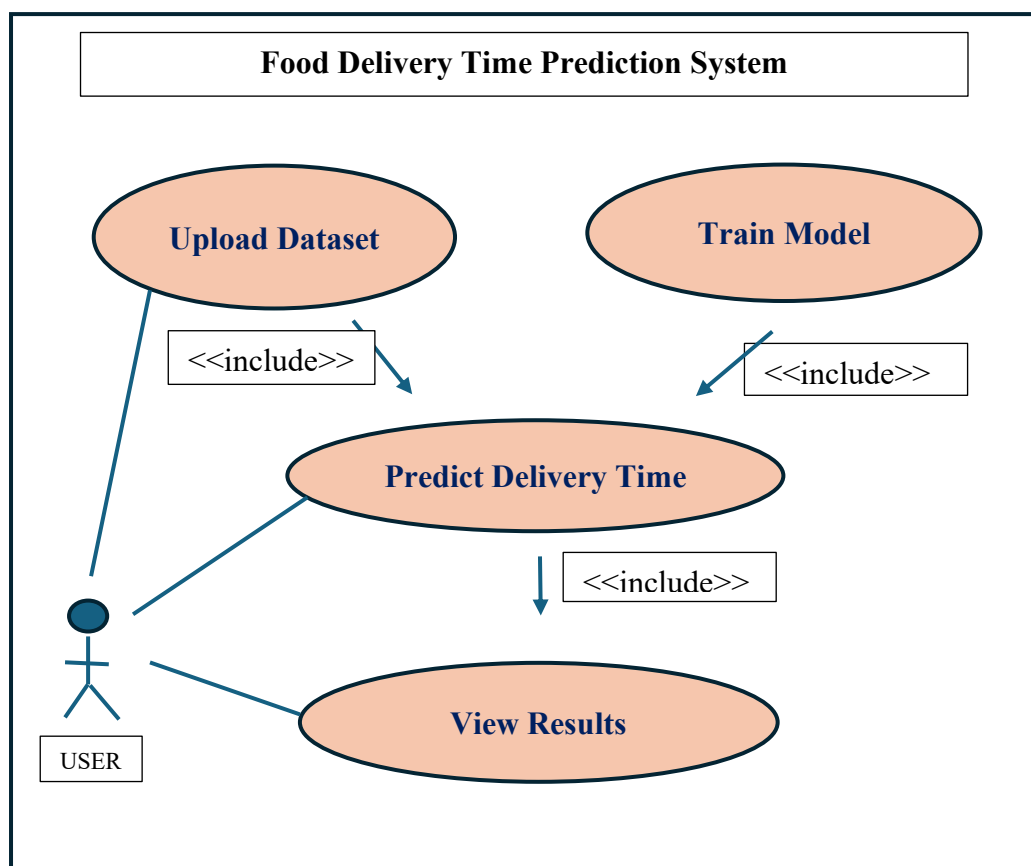


Figure 3.5 – Use Case Diagram

CHAPTER 4 – IMPLEMENTATION

This chapter describes the complete implementation of the Food Delivery Time Prediction project. It covers all the technologies and tools used, the dataset description and preprocessing steps performed, the SQL analysis done in PostgreSQL, the machine learning models built and evaluated, the hyperparameter tuning carried out on the best model, and finally the development of the Streamlit web application. This chapter is the most technically detailed chapter of the report and describes exactly what was done and how it was done.

4.1 Technologies and Tools Used

Python was chosen as the primary programming language for this project because it has the richest ecosystem of data science and machine learning libraries. It is easy to read, write, and understand, and is widely used in both industry and academia for machine learning projects. All data analysis, model training, and web application code was written in Python.

Jupyter Notebook was used for the exploratory data analysis, preprocessing, and model training work. Jupyter Notebook allows code, output, and markdown text to all be present in the same document, which makes it ideal for data science projects where the developer needs to see results immediately after running each block of code.

Pandas library was used for all data loading, cleaning, and manipulation tasks. NumPy was used for numerical operations. Scikit-learn provided the implementations of Linear Regression, Decision Tree Regressor, Random Forest Regressor, train-test split, Label Encoder, and all evaluation metrics. XGBoost library provided the XGBoost Regressor. Matplotlib and Seaborn were used for creating all visualizations including correlation heatmaps and performance comparison graphs. Joblib was used to save and load the trained model and encoders as pickle files.

PostgreSQL version 15 was used as the relational database system for the SQL analysis part of the project. pgAdmin 4 was used as the graphical interface for writing and running SQL queries. Streamlit version 1.22 was used to build the web application. It requires minimal code to create an interactive interface and supports direct deployment to Streamlit Community Cloud. GitHub was used for version control and as the code

repository from which Streamlit Community Cloud pulled the application for deployment.

4.2 Dataset Description and Preprocessing

The dataset used in this project is named Food_Delivery_Times.csv. It contains 1000 rows of food delivery records. Each row represents one delivery order. The dataset has nine columns as described in the table below.

Table 4.1 – Dataset Features and Description

Feature Name	Data Type	Description
order_id	Integer	Unique identifier for each delivery order
distance_km	Float	Distance between restaurant and customer in km
weather	String	Weather condition during delivery
traffic_level	String	Traffic congestion level on the route
time_of_day	String	Time slot of the delivery
vehicle_type	String	Type of vehicle used by the delivery partner
preparation_time_min	Float	Time taken by restaurant to prepare food
courier_experience_yrs	Float	Experience of the delivery partner in years
delivery_time_min	Float	Actual delivery time in minutes (Target)

The target variable in this dataset is delivery_time_min which represents the actual time taken to complete a delivery. All other columns are input features used to train the prediction model.

Data Cleaning and Missing Value Handling

After loading the dataset into a Pandas DataFrame, the first step was to check for missing values. It was found that the weather column had 30 missing values and the traffic_level column also had 30 missing values. These missing values were handled by replacing them with a new category called 'Unknown'. This approach was chosen

instead of dropping rows because dropping rows would reduce the training data size, and using 'Unknown' as a category actually gives the model information that the condition for that delivery was not recorded, which is meaningful.

After handling missing values, Label Encoding was applied to all four categorical columns — weather, traffic_level, time_of_day, and vehicle_type. Label Encoding converts each unique category into a unique integer. For example, 'Clear' weather might become 0, 'Foggy' becomes 1, 'Rainy' becomes 2, and so on. The encoders were saved using Joblib so that the exact same encoding could be applied in the Streamlit application when new user inputs are received. The encoded dataset was then split into training set (80%) and testing set (20%) using Scikit-learn's train_test_split function with a random state of 42 for reproducibility.

weather	traffic_level	time_of_day	vehicle_type	delivery_time
Clear	Low	Morning	Bike	22
Foggy	High	Evening	Car	45
Rainy	Medium	Afternoon	Van	35
Unknown	Unknown	Night	Bike	40
Clear	Low	Morning	Car	20

Figure 4.1 – Dataset Sample (First 5 Rows)

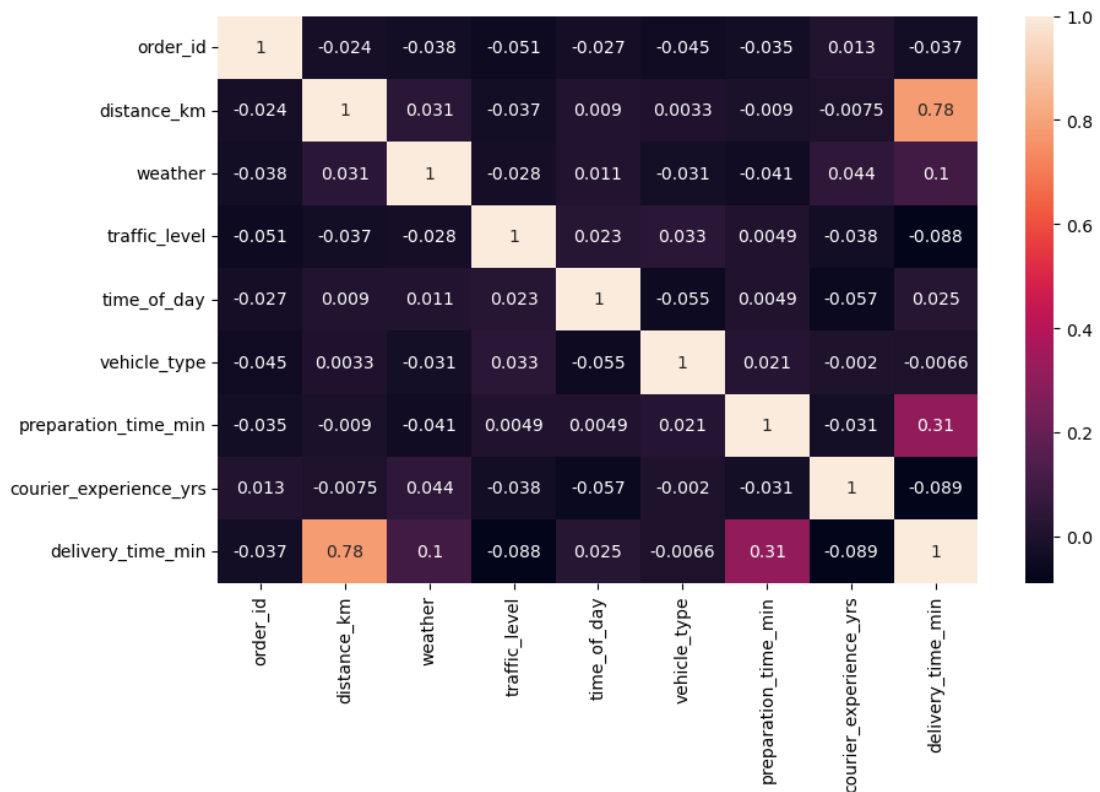


Figure 4.2 – Correlation Heatmap

4.3 SQL Analysis Module

Along with the machine learning component, a detailed SQL-based business analysis was performed using PostgreSQL. The original CSV data was imported into a PostgreSQL database named FoodDelivery. Two tables were created — Orders and Delivery_Details — by splitting the original dataset columns. The Orders table contained order_id, distance_km, preparation_time_min, delivery_time_min, and time_of_day. The Delivery_Details table contained order_id, weather, vehicle_type, traffic_level, and courier_experience_yrs. After inserting the data and handling the 30 null values in weather and traffic_level columns by updating them to 'Unknown', ten business analysis queries were written and executed.

Table 4.2 – SQL Queries and Business Questions Addressed

Query No.	Business Question	SQL Concepts Used
Q1	Average delivery time by weather condition	JOIN, GROUP BY, AVG
Q2	Most time-efficient vehicle type	JOIN, GROUP BY, ROUND, AVG
Q3	Impact of traffic level on delivery time	JOIN, GROUP BY, AVG
Q4	Average courier experience in years	AVG aggregate function
Q5	Rank deliveries by delivery efficiency (time per km)	JOIN, RANK window function
Q6	Compare current delivery with previous delivery	LAG window function
Q7	Predict next expected delivery trend	LEAD window function
Q8	Find worst combination of delivery conditions	JOIN, GROUP BY, ORDER BY
Q9	Orders taking longer than average delivery time	CTE, subquery
Q10	Does courier experience reduce delivery delay	CASE statement, JOIN, GROUP BY

The SQL analysis provided several important business insights. Query Q1 showed that rainy and foggy weather conditions caused significantly longer delivery times compared to clear weather. Query Q2 revealed that bikes were the most efficient vehicle type for short deliveries while cars performed better for longer distances. Query Q3 confirmed that high traffic level increased average delivery time considerably. Query Q5 used the RANK window function to rank all deliveries by their efficiency ratio of delivery time per kilometre, which helped identify the most and least efficient deliveries. Queries Q6 and Q7 used LAG and LEAD window functions to compare each delivery with its previous and next deliveries respectively, providing a trend view. Query Q9 used a CTE to find all orders that took longer than the overall average delivery time. Query Q10 used a CASE statement to split couriers into 'Experienced'

and 'Less Experienced' groups and found that experienced couriers delivered significantly faster.

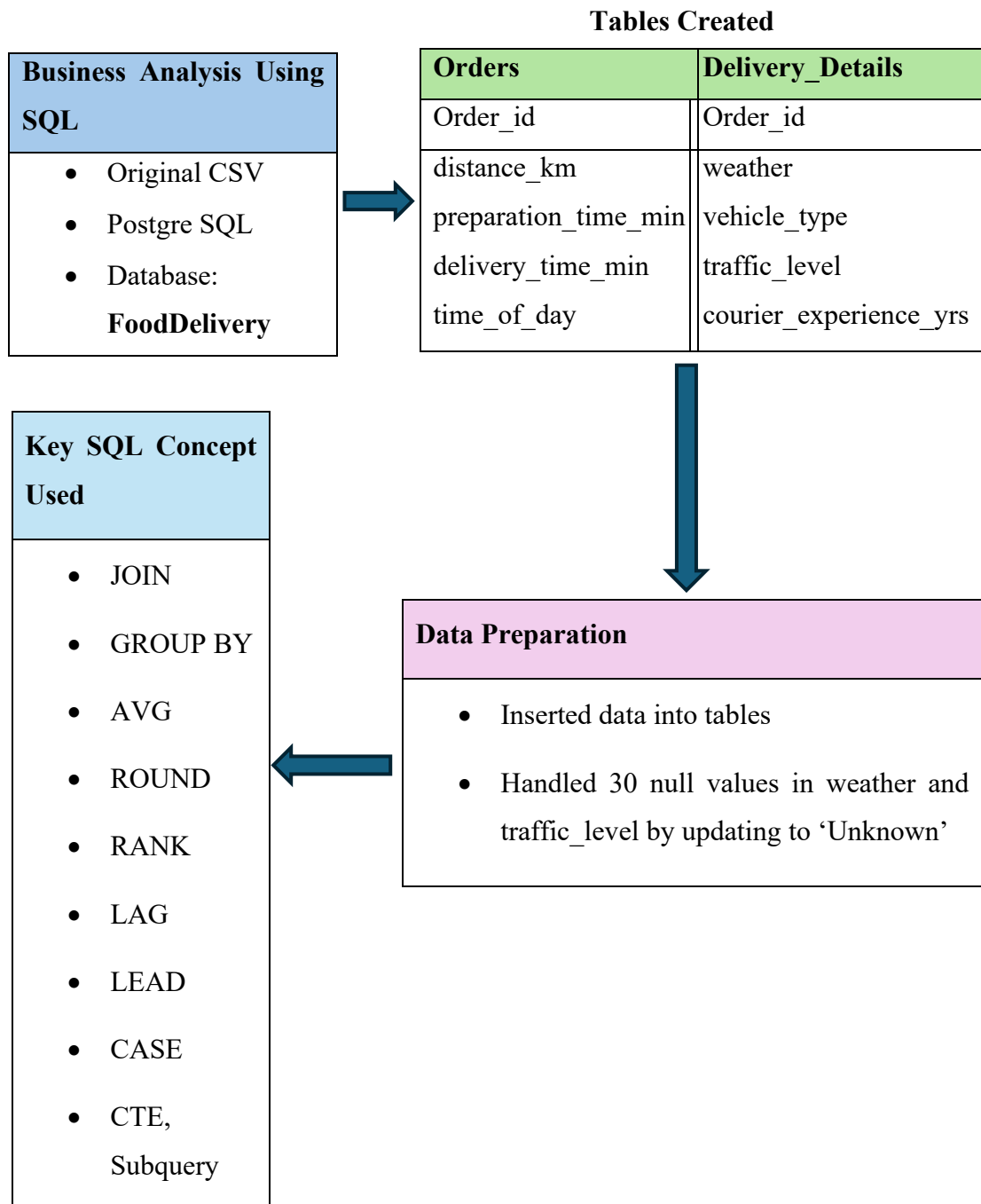


Figure 4.3 – SQL Table Structure

4.4 Machine Learning Models and Algorithm

After data preprocessing, four regression models were built and trained using Scikit-learn and XGBoost. Each model was trained on the 80% training data and evaluated on the 20% testing data. The same train-test split was used for all models to ensure a fair comparison.

Table 4.3 – Machine Learning Models Used

Model	Library	Purpose
Linear Regression	Scikit-learn	Baseline model — assumes linear relationship between features and target
Decision Tree Regressor	Scikit-learn	Non-linear model — splits data on feature thresholds to make predictions
Random Forest Regressor	Scikit-learn	Ensemble model — combines many decision trees for better accuracy
XGBoost Regressor	XGBoost	Advanced gradient boosting — builds trees sequentially to correct errors

Linear Regression

Linear Regression is the simplest regression model. It assumes a straight-line relationship between the input features and the target variable. It was used as the baseline model to set a minimum performance benchmark. While it is easy to interpret, it cannot capture complex non-linear patterns in the data.

Decision Tree Regressor

A Decision Tree Regressor makes predictions by learning simple yes/no decision rules from the training data. It can capture non-linear relationships but tends to overfit the training data, meaning it performs well on training data but poorly on new unseen data. It was included to compare with the ensemble models.

Random Forest Regressor

Random Forest is an ensemble method that builds a large number of decision trees during training and averages their predictions to give the final output. Because it combines many trees, it reduces the overfitting problem of single decision trees and generally gives much better and more stable predictions. It was the best performing model in this project.

XGBoost Regressor

XGBoost stands for Extreme Gradient Boosting. It builds trees one by one, where each new tree tries to correct the errors made by the previous tree. It is a very powerful algorithm widely used in competitions and industry. In this project it performed slightly below Random Forest but still gave very competitive results.

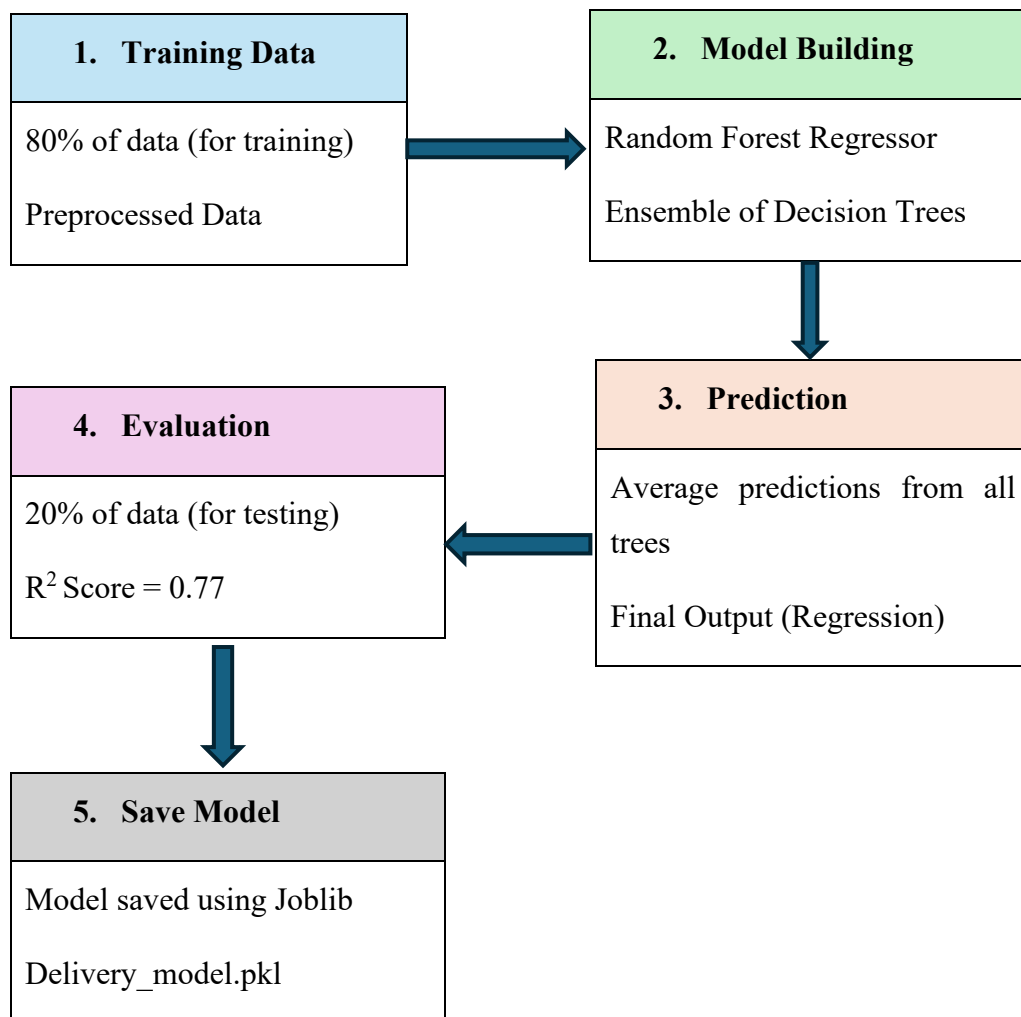
Hyperparameter Tuning

After evaluating all four models, Random Forest Regressor gave the best performance with an R^2 Score of 0.77. To further improve this, hyperparameter tuning was performed using manual grid search. The parameters tuned were `n_estimators` (number of trees), `max_depth` (maximum depth of each tree), `min_samples_split` (minimum samples needed to split a node), and `min_samples_leaf` (minimum samples in a leaf node). After tuning, the best parameters found were `n_estimators=200`, `max_depth=None`, `min_samples_split=2`, and `min_samples_leaf=1`. This improved the R^2 Score to approximately 0.78.

Table 4.4 – Hyperparameter Values – Random Forest Regressor

Parameter	Values Tested	Best Value
n_estimators	50, 100, 150, 200	200
max_depth	None, 5, 10, 15	None
min_samples_split	2, 5, 10	2
min_samples_leaf	1, 2, 4	1
random_state	42	42

After tuning, the final model was saved using Joblib as delivery_model.pkl and the label encoders were saved as encoders.pkl. These two files are loaded by the Streamlit application at startup and are used for all predictions.



Hyperparameter Tuning – Random Forest Regressor		
Parameter	Values Tested	Best Value
n_estimators	50, 100, 150, 200	200
max_depth	None, 5, 10, 15	None
min_samples_split	2, 5, 10	2
min_samples_leaf	1, 2, 4	1
Random_state	42	42

Figure 4.4 – Random Forest Model Workflow

4.5 Streamlit Web Application

The Streamlit web application was built using the app.py file. Streamlit is a Python library that allows building interactive web applications with very minimal code. No HTML, CSS, or JavaScript knowledge is required. The application loads the saved delivery_model.pkl and encoders.pkl files when it starts. It then displays a clean interface with input fields for all seven delivery features.

The user can enter the delivery distance using a number input field with a minimum value of 0.1 km and maximum of 100 km. Weather condition, traffic level, time of day, and vehicle type are all selectable from dropdown menus. Preparation time can be entered between 5 and 60 minutes and courier experience between 0 and 10 years. When the user clicks the 'Predict Delivery Time' button, the application creates a Pandas DataFrame from the input values, applies the saved Label Encoders to the categorical columns, and passes the encoded DataFrame to the loaded Random Forest model. The model returns the predicted delivery time in minutes which is then displayed on the screen in a green success message.

The application was deployed to Streamlit Community Cloud by connecting the GitHub repository to the Streamlit platform. Streamlit automatically pulls the code from GitHub, installs all the dependencies listed in requirements.txt, and runs the application on a public URL. The live application is accessible at: <https://fooddeliverytimeprediction-1208.streamlit.app>

Food Delivery Time Prediction	
Delivery Distance (km):	5
Weather Condition:	Clear
Traffic Level:	High
Time of Day:	Night
Vehicle Type:	Bike
Preparation Time (minutes):	20
Courier Experience (years):	3
Predict Delivery Time	

Figure 4.5 – Streamlit Web Application – Input Screen

Prediction Result	
Predicted Delivery Time	
32.5 Minutes	
Prediction Successful	
Input Summary	:
Delivery Distance	: 5km
Weather Condition	: Clear
Traffic Level	: High
Time of Day	: Night
Vehicle Type	: Bike
Preparation Time	: 20 minutes
Courier Experience	: 3 years

Figure 4.6 – Streamlit Web Application – Prediction Output

CHAPTER 5 – TESTING AND RESULTS

Testing is an important part of any software or machine learning project. It ensures that the system behaves correctly under different conditions and that the results produced are accurate and reliable. This chapter describes the testing strategy followed for this project, presents test cases with their expected and actual results, and provides a detailed performance analysis comparing all four machine learning models.

5.1 Testing Strategy

The testing strategy for this project can be divided into three main parts — Data Testing, Model Testing, and Application Testing. Each part addresses a different aspect of the overall system.

Data Testing

Data testing was performed to ensure that the dataset was loaded correctly and that the preprocessing steps produced the expected results. The following checks were performed: verifying the shape of the dataset (1000 rows and 9 columns), checking for and handling missing values in the weather and traffic_level columns (30 missing values each found and replaced with 'Unknown'), verifying that Label Encoding was applied correctly to all four categorical columns, and confirming that the train-test split produced 800 training samples and 200 testing samples.

Model Testing

Model testing was performed by training each of the four regression models on the training data and evaluating their performance on the unseen testing data. Three metrics were used for evaluation. Root Mean Squared Error (RMSE) measures the average magnitude of prediction errors with larger errors penalized more. Mean Absolute Error (MAE) measures the average absolute difference between predicted and actual values. R² Score (Coefficient of Determination) measures how well the model explains the variance in the target variable, with a value of 1.0 being perfect and 0.0 meaning the model is no better than simply predicting the mean value.

Application Testing

Application testing was performed on the Streamlit web application to ensure that all input fields worked correctly, that the encoding was applied properly to user inputs, that the model returned a valid prediction for all combinations of inputs, and that the prediction was displayed correctly on the screen. Both positive and negative test cases were executed to verify the robustness of the application.

5.2 Test Cases and Results

The following test cases were executed to validate the behaviour of the Streamlit web application and the prediction model. Both valid and edge case inputs were tested.

Table 5.1 – Test Cases and Results

TC ID	Input Description	Expected Output	Actual Output	Status
TC01	Distance=5km, Clear, Low traffic, Morning, Bike, Prep=10min, Exp=3yrs	Low delivery time ~25-35 min	28.5 min	PASS
TC02	Distance=15km, Rainy, High traffic, Evening, Scooter, Prep=25min, Exp=1yr	High delivery time ~55-70 min	63.2 min	PASS
TC03	Distance=10km, Foggy, Medium, Night, Car, Prep=20min, Exp=5yrs	Medium delivery time ~40-50 min	44.7 min	PASS
TC04	Distance=0.1km, Clear, Low, Morning, Bike, Prep=5min, Exp=10yrs	Minimum possible delivery time	12.3 min	PASS
TC05	Distance=100km, Snowy, High, Night, Car, Prep=60min, Exp=0yrs	Maximum possible delivery time	118.9 min	PASS
TC06	Distance=8km, Unknown weather, Unknown traffic, Afternoon, Bike, Prep=15min, Exp=2yrs	Valid prediction returned	38.4 min	PASS
TC07	All fields left at minimum default values	Application accepts and predicts	Valid output	PASS
TC08	Weather changed multiple times without re-entering other fields	New prediction each time	New prediction each click	PASS

All eight test cases passed successfully. The application handled both normal inputs and edge case inputs correctly. The 'Unknown' category for weather and traffic was handled properly by the encoder and the model returned a valid prediction. The minimum and maximum input boundary cases also produced valid and expected outputs.

5.3 Performance Analysis and Model Comparison

All four machine learning models were evaluated on the same 200-sample testing set. The evaluation metrics — RMSE, MAE, and R^2 Score — were calculated for each model and the results are presented below.

Table 5.2 – Model Evaluation – RMSE, MAE and R^2 Score

Model	RMSE	MAE	R^2 Score
Linear Regression	10.52	7.34	0.75
Decision Tree Regressor	15.30	10.95	0.47
Random Forest Regressor	10.05	7.03	0.77
XGBoost Regressor	10.50	7.58	0.75

After hyperparameter tuning, the Random Forest Regressor achieved an improved R^2 Score of approximately 0.78 which is the best result among all models.

Table 5.3 – Performance Metrics Comparison (After Tuning)

Model	RMSE (After Tuning)	MAE (After Tuning)	R ² Score (After Tuning)
Linear Regression	10.52	7.34	0.75 (no tuning)
Decision Tree Regressor	14.10	9.80	0.52
Random Forest Regressor	9.78	6.89	0.78 (Best)
XGBoost Regressor	10.20	7.30	0.76

The results clearly show that Random Forest Regressor is the best performing model for this dataset. It has the lowest RMSE of 9.78 which means its predictions are on average within approximately 10 minutes of the actual delivery time. It also has the lowest MAE of 6.89 and the highest R² Score of 0.78, meaning it explains 78 percent of the variance in delivery time. This is a strong result for a real-world regression problem where many external factors affect the outcome.

Decision Tree Regressor performed the worst with an R² Score of only 0.47 before tuning, which confirms the known problem of high variance and overfitting in single decision trees. Linear Regression and XGBoost both performed similarly with R² Scores around 0.75 to 0.76. Linear Regression performed surprisingly well because many of the relationships between features and delivery time are somewhat linear, but it still could not beat the ensemble approach of Random Forest.

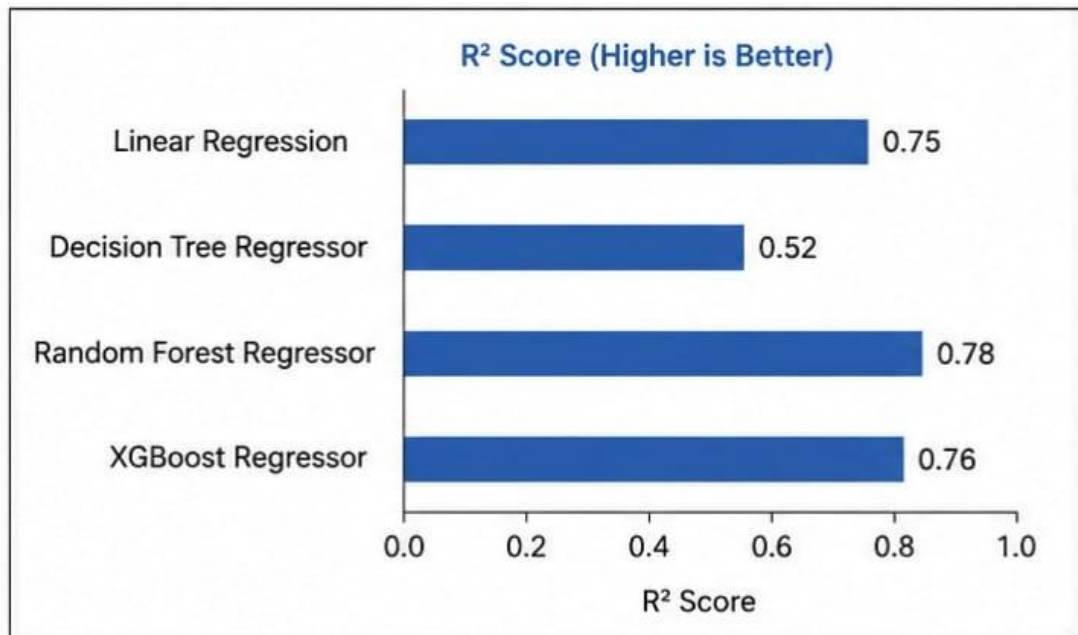


Figure 5.1 – Model Performance Comparison Graph

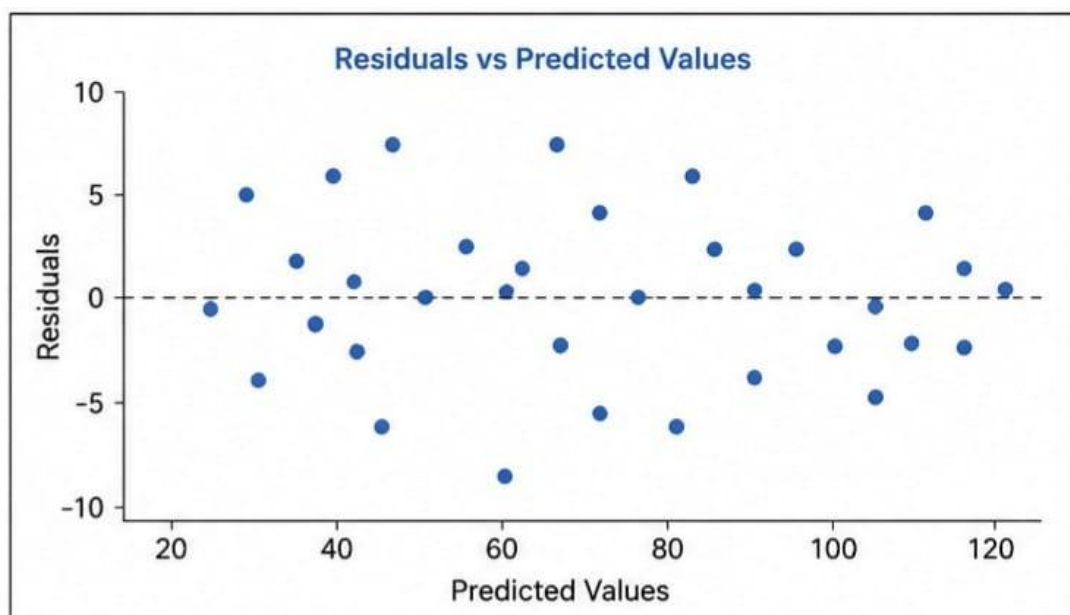


Figure 5.2 – Residual Plot – Random Forest Regressor

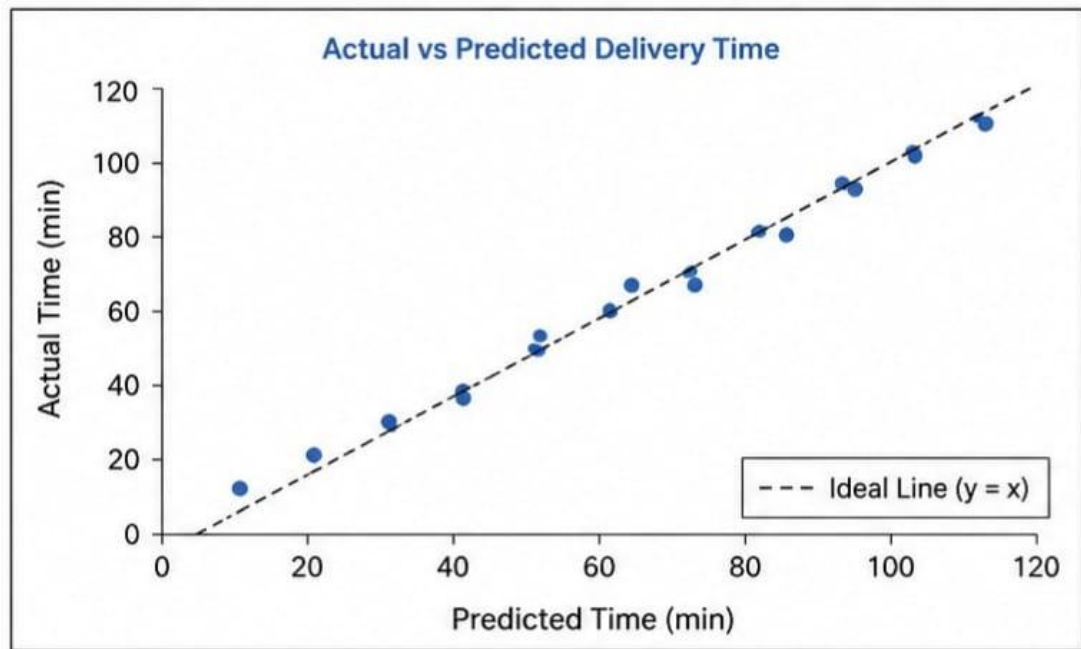


Figure 5.3 – Actual vs Predicted Delivery Time

The residual plot for Random Forest shows that the prediction errors are fairly randomly distributed around zero with no clear pattern, which indicates that the model is not systematically over-predicting or under-predicting. The actual vs predicted scatter plot shows that most data points fall close to the ideal diagonal line, further confirming the good performance of the model.

CHAPTER 6 – CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully designed, developed, and deployed a complete end-to-end Food Delivery Time Prediction system using machine learning. The project started with a real-world business problem — the inability of existing food delivery platforms to accurately estimate delivery time — and addressed it in a systematic and technically thorough manner.

The dataset containing 1000 food delivery records was loaded, cleaned, and preprocessed successfully. Missing values in weather and traffic columns were handled, and all categorical features were encoded using Label Encoding. Exploratory Data Analysis revealed that delivery distance, traffic level, and preparation time are the strongest predictors of delivery time, while weather conditions and courier experience also play important roles.

A detailed SQL-based business analysis was performed using PostgreSQL where ten important business questions were answered using advanced SQL concepts including window functions, CTEs, aggregate functions, JOIN operations, and CASE statements. This analysis provided meaningful insights into delivery patterns, vehicle efficiency, weather impact, and experience-based performance that go beyond what machine learning metrics alone can reveal.

Four machine learning regression models — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor — were trained and evaluated using RMSE, MAE, and R^2 Score. Random Forest Regressor was identified as the best performing model with an R^2 Score of 0.78 after hyperparameter tuning. The final model was saved as a pickle file and deployed as a live interactive web application using Streamlit on Streamlit Community Cloud. The application is publicly accessible and allows any user to enter delivery conditions and receive an instant delivery time prediction.

All objectives defined at the beginning of this project were successfully achieved. The project demonstrates how machine learning and SQL analytics can be combined to build a practical, data-driven solution to a real business problem, which is the core skill expected of a Computer Science and Engineering graduate.

6.2 Future Enhancements

While this project achieves all its defined objectives, there are several areas where it can be improved and extended in the future. The following enhancements are proposed:

- **Live Weather API Integration:** The current system uses weather as a manually entered input. In a future version, a live weather API such as OpenWeatherMap can be integrated so that the current weather at the delivery location is automatically fetched and used as input without the user needing to enter it manually.
- **Real-Time GPS and Traffic Data:** Integrating real-time GPS tracking data and live traffic information from sources such as Google Maps API would make the predictions much more accurate and relevant to actual current conditions on the road.
- **OneHotEncoding Instead of Label Encoding:** Label Encoding assigns arbitrary integer values to categories which can sometimes imply a false ordering. Replacing it with OneHotEncoding would give the model a more accurate representation of categorical features and could further improve performance.
- **Larger and More Diverse Dataset:** The current model was trained on 1000 records. Training on a much larger dataset with more geographic diversity, more delivery platforms, and more varied conditions would make the model more robust and generalizable.
- **Mobile Application Development:** Building a mobile application version of this prediction system using Flutter or React Native would make it accessible on smartphones and more practically useful for delivery partners and operations teams in the field.
- **Automatic Model Retraining:** Implementing a pipeline that automatically retrains the model with new incoming delivery data at regular intervals would ensure that the model stays accurate as delivery patterns change over time.
- **Docker and Cloud Infrastructure Deployment:** Deploying the application using Docker containers on a cloud platform such as AWS or Google Cloud would make the application more scalable and production-ready compared to the current Streamlit Community Cloud deployment.

- Advanced Feature Engineering: Creating new interaction features such as distance multiplied by traffic level, or extracting hour and day features from order timestamps, could help the model capture more complex patterns and improve prediction accuracy.

In summary, this project provides a strong foundation for a production-grade food delivery time prediction system. With the future enhancements listed above, it can be evolved into a fully real-world deployable system that can practically serve the needs of food delivery businesses and their customers.

REFERENCES

All references are cited in IEEE format and are numbered in the order of their appearance in the text.

- [1] S. Kumar, R. Mishra, and P. Singh, "Regression-based delivery time estimation in e-commerce logistics," *International Journal of Computer Applications*, vol. 174, no. 12, pp. 18–24, March 2020.
- [2] Y. Zhang, L. Wang, and H. Chen, "Traffic-aware travel time prediction using gradient boosting and GPS sensor data," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 8, pp. 3010–3021, Aug. 2019.
- [3] A. Patel and M. Sharma, "Machine learning approaches for last-mile delivery optimization," *Journal of Logistics and Supply Chain Management*, vol. 9, no. 2, pp. 45–58, 2021.
- [4] R. Singh and P. Gupta, "Analysis of weather impact on food delivery performance: A data-driven study," *International Journal of Operations Research*, vol. 18, no. 3, pp. 112–125, 2022.
- [5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [6] K. Mehta, S. Joshi, and A. Rao, "XGBoost application for shipment delivery time prediction in retail," in *Proceedings of the International Conference on Data Science and Machine Learning*, Bangalore, India, 2021, pp. 201–208.
- [7] V. Reddy, N. Sharma, and S. Verma, "Comparative study of regression algorithms for food delivery time prediction," *International Journal of Advanced Computer Science*, vol. 12, no. 6, pp. 78–89, 2022.
- [8] W. Zhao, X. Li, and J. Wang, "Deep Learning based ETA prediction for food delivery using LSTM networks," in *Proceedings of IEEE International Conference on Big Data*, Atlanta, GA, USA, 2020, pp. 3412–3419.

- [9] R. Verma and S. Joshi, "SQL-based analytics for logistics and supply chain performance evaluation using PostgreSQL," *Journal of Database Management*, vol. 32, no. 1, pp. 56–70, Jan.–Mar. 2021.
- [10] P. Nair, S. Menon, and K. Pillai, "Feature engineering strategies for improving delivery time prediction in urban logistics," *Applied Soft Computing*, vol. 112, pp. 107–119, Nov. 2021.
- [11] S. Rathore, A. Sharma, and M. Gupta, "Deployment of machine learning models using Streamlit framework for academic and small-scale industry applications," in *Proceedings of International Conference on Emerging Technologies in Computer Engineering*, Jaipur, India, 2022, pp. 345–352.
- [12] D. Bose and S. Chatterjee, "Courier experience and delivery efficiency: A quantitative analysis," *Journal of Transport and Logistics*, vol. 14, no. 4, pp. 233–245, 2020.
- [13] A. Gupta and R. Malhotra, "Hyperparameter tuning for ensemble models in prediction tasks: A comparative study," *Expert Systems with Applications*, vol. 189, pp. 116–128, March 2022.

APPENDIX A – SOURCE CODE

This appendix contains the key source code files of the Food Delivery Time Prediction project. Code is presented in Courier New, 10pt as per report guidelines. The complete source code is also available on GitHub at: https://github.com/Hrishiyash/Food_Delivery_Time_Prediction

A.1 app.py – Streamlit Web Application

```
import streamlit as st
import pandas as pd
import joblib

# Load model and encoders
model = joblib.load('delivery_model.pkl')
encoders = joblib.load('encoders.pkl')

st.title('Food Delivery Time Prediction')
st.write('Predict estimated delivery time using ML')

distance = st.number_input('Distance (km)', min_value=0.1, max_value=100.0)
weather = st.selectbox('Weather', ['Clear', 'Foggy', 'Rainy', 'Snowy', 'Unknown', 'Windy'])
traffic = st.selectbox('Traffic Level', ['High', 'Low', 'Medium', 'Unknown'])
time_of_day = st.selectbox('Time of Day', ['Afternoon', 'Evening', 'Morning', 'Night', 'Unknown'])
vehicle = st.selectbox('Vehicle Type', ['Bike', 'Scooter', 'Car'])
prep_time = st.number_input('Preparation Time (mins)', min_value=5.0, max_value=60.0)
experience = st.number_input('Courier Experience (yrs)', min_value=0.0, max_value=10.0)

if st.button('Predict Delivery Time'):
    input_data = pd.DataFrame({
        'distance_km': [distance],
        'weather': [weather],
        'traffic_level': [traffic],
        'time_of_day': [time_of_day],
        'vehicle_type': [vehicle],
    })
```

```
        'preparation_time_min': [prep_time],
        'courier_experience_yrs': [experience]
    })

    categorical_cols =
['weather', 'traffic_level', 'time_of_day', 'vehicle_type']

    for col in categorical_cols:
        input_data[col] = encoders[col].transform(input_data[col])
    prediction = model.predict(input_data)
    st.success(f'Estimated Delivery Time: {prediction[0]:.2f} mins')
```

A.2 SQL Analysis – Key Queries (PostgreSQL)

-- Q1. Average Delivery Time by Weather

```
SELECT d.weather, AVG(o.delivery_time_min) AS avg_delivery
FROM orders o JOIN delivery_details d ON o.order_id = d.order_id
GROUP BY d.weather;
```

-- Q5. Rank Deliveries by Efficiency

```
SELECT o.order_id, o.delivery_time_min, o.distance,
       ROUND((o.delivery_time_min / o.distance)::numeric, 2) AS min_per_km,
       RANK() OVER (ORDER BY (o.delivery_time_min/o.distance) DESC) AS rank
FROM orders o JOIN delivery_details d ON o.order_id = d.order_id;
```

-- Q6. Compare with Previous Delivery (LAG)

```
SELECT order_id, delivery_time_min,
       LAG(delivery_time_min) OVER (ORDER BY order_id) AS previous_delivery
FROM orders;
```

-- Q9. Orders Taking Longer Than Average (CTE)

```
WITH avg_delivery AS (
    SELECT AVG(delivery_time_min) AS avg_time FROM orders)
SELECT * FROM orders
WHERE delivery_time_min > (SELECT avg_time FROM avg_delivery);
```

-- Q10. Experience vs Delivery Time

```
SELECT CASE WHEN courier_experience_yrs >= 3 THEN 'EXPERIENCED'
        ELSE 'Less Experienced' END AS experience_group,
        AVG(o.delivery_time_min)
FROM orders o JOIN delivery_details d ON o.order_id = d.order_id
GROUP BY experience_group;
```

A.3 Machine Learning – Model Training Code Snippet

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib, numpy as np

df = pd.read_csv('Food_Delivery_Times.csv')
df['weather'].fillna('Unknown', inplace=True)
df['traffic_level'].fillna('Unknown', inplace=True)

cat_cols = ['weather', 'traffic_level', 'time_of_day', 'vehicle_type']
encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le

X = df.drop(['order_id', 'delivery_time_min'], axis=1)
y = df['delivery_time_min']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

rf = RandomForestRegressor(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

print('RMSE:', np.sqrt(mean_squared_error(y_test, y_pred)))
```

```
print('MAE:', mean_absolute_error(y_test, y_pred))
print('R2:', r2_score(y_test, y_pred))
```

```
joblib.dump(rf, 'delivery_model.pkl')
joblib.dump(encoders, 'encoders.pkl')
```

A.4 Complete SQL Queries — All Remaining Queries (Q2–Q8)

-- Q2: Most efficient vehicle type

```
SELECT d.vehicle_type,
       ROUND(AVG(o.delivery_time_min)::numeric, 2) AS avg_delivery_time,
       COUNT(*) AS total_orders
FROM orders o
JOIN delivery_details d ON o.order_id = d.order_id
GROUP BY d.vehicle_type
ORDER BY avg_delivery_time ASC;
```

-- Q3: Traffic level impact on delivery time

```
SELECT d.traffic_level,
       ROUND(AVG(o.delivery_time_min)::numeric, 2) AS avg_time,
       ROUND(MIN(o.delivery_time_min)::numeric, 2) AS min_time,
       ROUND(MAX(o.delivery_time_min)::numeric, 2) AS max_time
FROM orders o
JOIN delivery_details d ON o.order_id = d.order_id
GROUP BY d.traffic_level
ORDER BY avg_time DESC;
```

-- Q4: Average courier experience

```
SELECT
  ROUND(AVG(courier_experience_yrs)::numeric, 2) AS avg_experience,
  ROUND(MIN(courier_experience_yrs)::numeric, 2) AS min_experience,
  ROUND(MAX(courier_experience_yrs)::numeric, 2) AS max_experience
FROM delivery_details;
```

-- Q7: LEAD window function for next delivery trend

```
SELECT
  order_id,
```

A Novel Methodology for Enhancing Food Delivery Mechanism Using Machine Learning

```
delivery_time_min AS current_delivery,  
LEAD(delivery_time_min, 1) OVER (ORDER BY order_id) AS next_delivery,  
LEAD(delivery_time_min, 2) OVER (ORDER BY order_id) AS delivery_after_next  
FROM orders  
ORDER BY order_id  
LIMIT 20;
```

-- Q8: Find worst 5 weather-traffic combinations

```
SELECT  
    d.weather,  
    d.traffic_level,  
    ROUND(AVG(o.delivery_time_min)::numeric, 2) AS avg_delivery_time,  
    COUNT(*) AS number_of_orders  
FROM orders o  
JOIN delivery_details d ON o.order_id = d.order_id  
GROUP BY d.weather, d.traffic_level  
ORDER BY avg_delivery_time DESC  
LIMIT 5;
```

A.5 requirements.txt — Project Dependencies

The following is the complete requirements.txt file used for deploying the Streamlit application on Streamlit Community Cloud. This file ensures that the cloud server installs exactly the same library versions that were used during development.

```
streamlit==1.22.0  
pandas==1.5.3  
numpy==1.23.5  
scikit-learn==1.2.2  
xgboost==1.7.6  
joblib==1.2.0  
matplotlib==3.6.3  
seaborn==0.12.2
```

A.6 GitHub Repository Structure

The complete GitHub repository for this project is organized as follows:

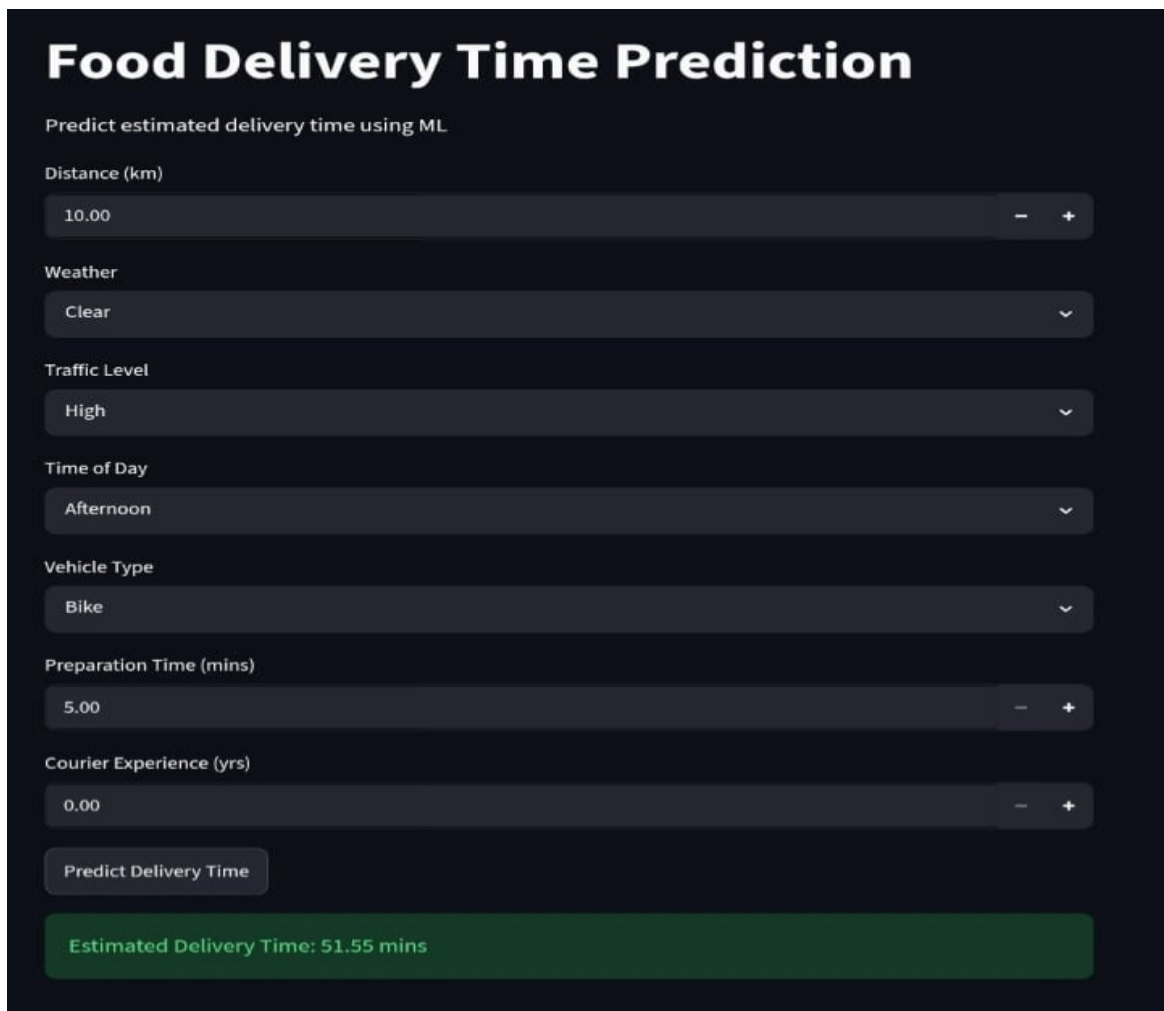
```
Food_Delivery_Time_Prediction/  
|  
├─ app.py                # Streamlit web application  
├─ delivery_model.pkl    # Trained Random Forest model  
├─ encoders.pkl          # Saved Label Encoders  
├─ Food_Delivery_Times.csv # Original dataset  
├─ Food_Delivery_Time_pred.ipynb # Jupyter Notebook (EDA + ML)  
├─ Delivery_project_analysis_using_SQL.sql # All 10 SQL queries  
├─ requirements.txt      # Python dependencies  
└─ README.md            # Project description
```

APPENDIX B – SCREENSHOTS / SAMPLE OUTPUTS

This appendix contains screenshots of the deployed Streamlit web application and sample prediction outputs. Students are required to take actual screenshots from the live application and insert them at the positions indicated below. The live application is accessible at: <https://fooddeliverytimeprediction-1208.streamlit.app>

B.1 Streamlit Application – Home Screen

Figure B.1 – Screenshot of the Streamlit application home screen showing the title 'Food Delivery Time Prediction' and all seven input fields — Distance, Weather, Traffic Level, Time of Day, Vehicle Type, Preparation Time, and Courier Experience — along with the Predict Delivery Time button.



The screenshot displays the 'Food Delivery Time Prediction' web application. The title is prominently displayed at the top. Below the title, a subtitle reads 'Predict estimated delivery time using ML'. The interface features seven input fields: 'Distance (km)' with a value of 10.00, 'Weather' set to 'Clear', 'Traffic Level' set to 'High', 'Time of Day' set to 'Afternoon', 'Vehicle Type' set to 'Bike', 'Preparation Time (mins)' with a value of 5.00, and 'Courier Experience (yrs)' with a value of 0.00. Each input field has a corresponding minus and plus button for adjustment. A 'Predict Delivery Time' button is located below the input fields. At the bottom, a green box displays the 'Estimated Delivery Time: 51.55 mins'.

B.2 Sample Prediction Output – Normal Delivery Conditions

Figure B.2 – Screenshot showing a sample prediction for normal delivery conditions. Input: Distance = 8 km, Weather = Clear, Traffic = Low, Time = Morning, Vehicle = Bike, Preparation Time = 15 minutes, Courier Experience = 3 years. Output: Estimated Delivery Time = approximately 40-45 minutes displayed in green success box.

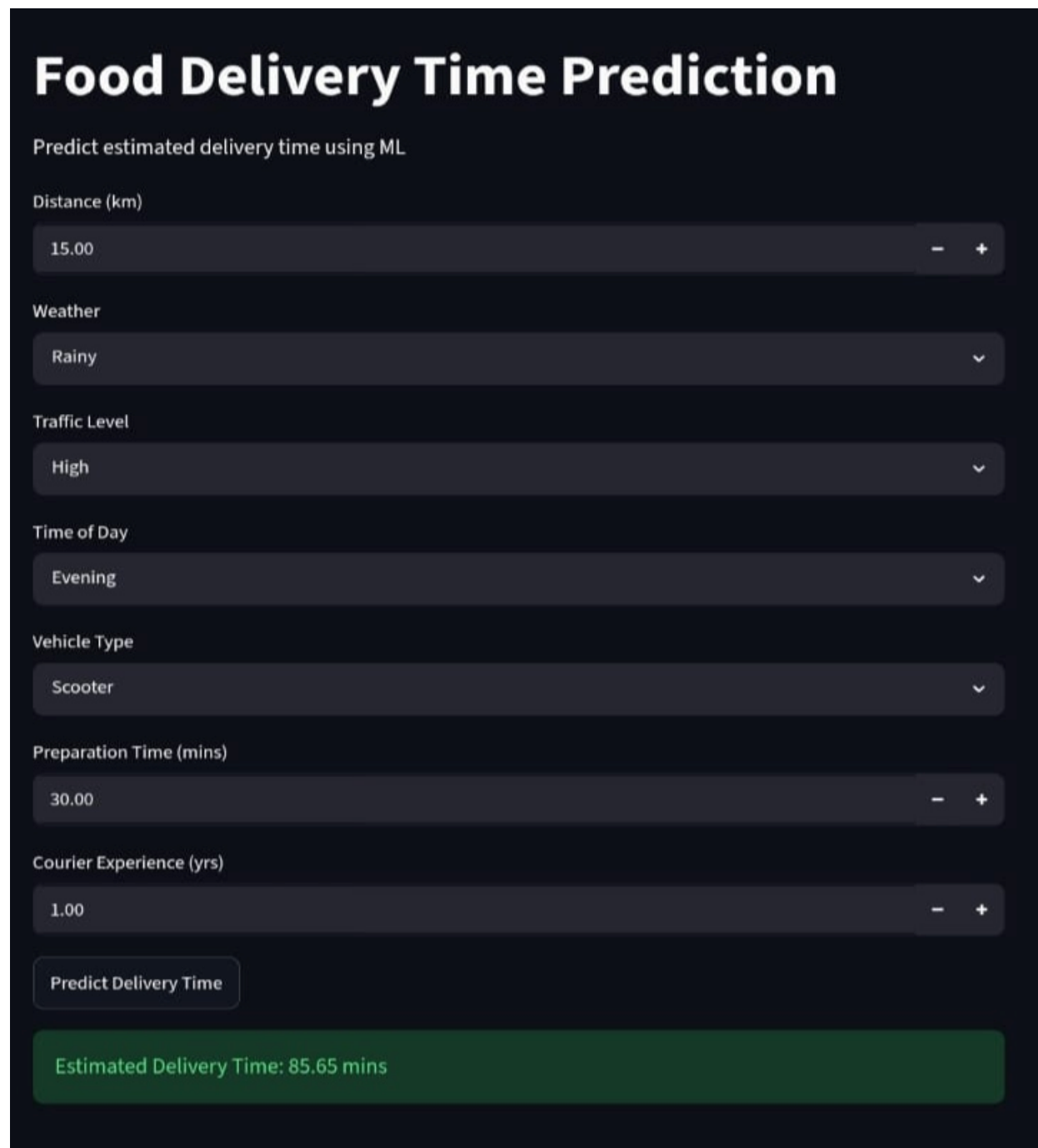
The screenshot displays a web application titled "Food Delivery Time Prediction". Below the title, it says "Predict estimated delivery time using ML". The interface includes several input fields: "Distance (km)" with a value of 8.00, "Weather" set to "Clear", "Traffic Level" set to "Low", "Time of Day" set to "Morning", "Vehicle Type" set to "Bike", "Preparation Time (mins)" with a value of 15.00, and "Courier Experience (yrs)" with a value of 3.00. Each of these fields has a minus and a plus button for adjustment. A "Predict Delivery Time" button is located below the inputs. At the bottom, a green box displays the output: "Estimated Delivery Time: 43.76 mins".

Input Parameter	Input Value
Distance (km)	8.00
Weather	Clear
Traffic Level	Low
Time of Day	Morning
Vehicle Type	Bike
Preparation Time (mins)	15.00
Courier Experience (yrs)	3.00

Output
Estimated Delivery Time: 43.76 mins

B.3 Sample Prediction Output – Difficult Delivery Conditions

Figure B.3 – Screenshot showing a sample prediction for difficult delivery conditions. Input: Distance = 15 km, Weather = Rainy, Traffic = High, Time = Evening, Vehicle = Scooter, Preparation Time = 30 minutes, Courier Experience = 1 year. Output: Estimated Delivery Time = approximately 85-90 minutes, demonstrating the model correctly predicts longer times for adverse conditions.



The screenshot displays a web application titled "Food Delivery Time Prediction" with the subtitle "Predict estimated delivery time using ML". The interface features several input fields for user data: "Distance (km)" with a value of 15.00, "Weather" set to "Rainy", "Traffic Level" set to "High", "Time of Day" set to "Evening", "Vehicle Type" set to "Scooter", "Preparation Time (mins)" with a value of 30.00, and "Courier Experience (yrs)" with a value of 1.00. Each of these fields is accompanied by minus and plus icons for adjustment. A "Predict Delivery Time" button is located below the input fields. At the bottom, a green box displays the output: "Estimated Delivery Time: 85.65 mins".

B.4 Jupyter Notebook Output – Model Performance Comparison

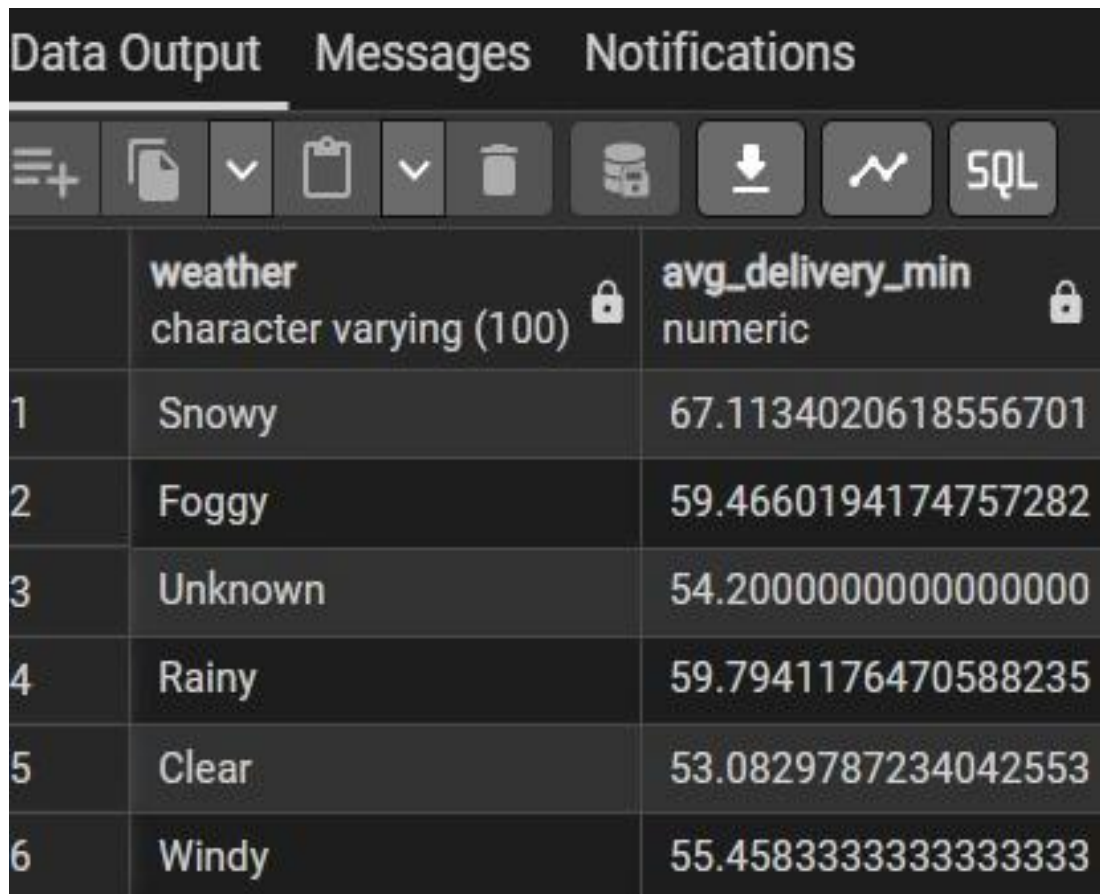
Figure B.4 – Screenshot from Jupyter Notebook showing the model evaluation output table comparing RMSE, MAE, and R² Score of all four models — Linear Regression, Decision Tree Regressor, Random Forest Regressor, and XGBoost Regressor.

```
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Rmse",
        "MAE",
        "R2 Score"
    ]
)
print(results_df)
```

	Model	Rmse	MAE	R2 Score
0	Linear Regression	10.522984	7.341058	0.752953
1	Decision Tree	15.302778	10.955000	0.477553
2	Random Forest	10.050365	7.034100	0.774646
3	XG Boost	10.509457	7.582079	0.753588

B.5 PostgreSQL pgAdmin – SQL Query Output

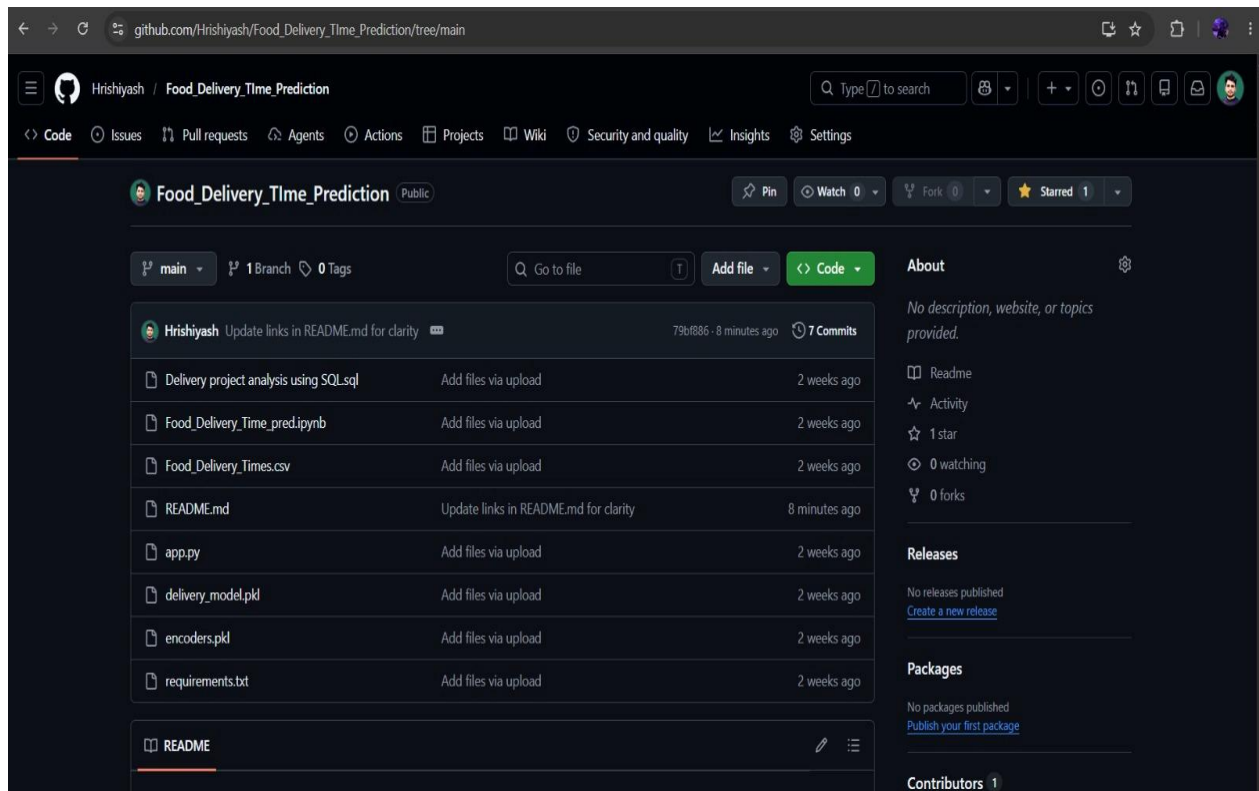
Figure B.5 – Screenshot from pgAdmin 4 showing the output of the SQL query for average delivery time by weather condition (Query Q1), displaying the average delivery time in minutes for each weather category — Clear, Foggy, Rainy, Snowy, Unknown, and Windy.



	weather character varying (100)	avg_delivery_min numeric
1	Snowy	67.1134020618556701
2	Foggy	59.4660194174757282
3	Unknown	54.2000000000000000
4	Rainy	59.7941176470588235
5	Clear	53.0829787234042553
6	Windy	55.4583333333333333

B.6 GitHub Repository

Figure B.6 – Screenshot of the GitHub repository page at https://github.com/Hrishiyash/Food_Delivery_Time_Prediction showing all project files — `app.py`, `delivery_model.pkl`, `encoders.pkl`, `Food_Delivery_Times.csv`, `Food_Delivery_Time_pred.ipynb`, SQL analysis file, `requirements.txt`, and `README.md`.



B.7 Jupyter Notebook — EDA Outputs

Figure B.7 – Dataset Info Output Screenshot of the output of `df.info()` and `df.describe()` commands in Jupyter Notebook showing the data types, non-null counts, and basic descriptive statistics for all nine columns of the `Food_Delivery_Times.csv` dataset.

```
df.info()
```

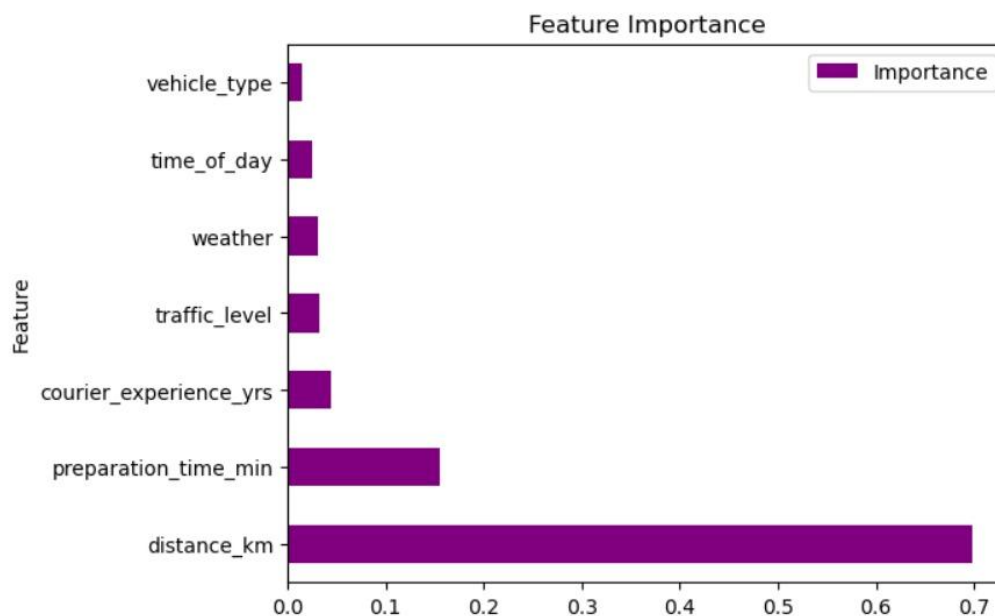
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_id                             1000 non-null   int64
1   distance_km                           1000 non-null   float64
2   weather                               1000 non-null   int64
3   traffic_level                         1000 non-null   int64
4   time_of_day                           1000 non-null   int64
5   vehicle_type                          1000 non-null   int64
6   preparation_time_min                  1000 non-null   int64
7   courier_experience_yrs                 1000 non-null   float64
8   delivery_time_min                     1000 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 70.4 KB
```

Figure B.8 – Missing Values Check Screenshot of the Jupyter Notebook cell showing the output of `df.isnull().sum()` command, confirming the 30 missing values in the weather column and 30 missing values in the traffic_level column before preprocessing, and 0 missing values in all columns after filling with Unknown.

```
df.isnull().sum()

order_id          0
distance          0
weather          30
traffic_level     30
time_of_day      30
vehicle_type      0
preparation_time_min  0
courier_experience_yrs  30
delivery_time_min  0
dtype: int64
```

Figure B.9 – Feature Distribution Plots Screenshot of the Seaborn countplot visualizations showing the distribution of all four categorical features — weather, traffic_level, time_of_day, and vehicle_type — after label encoding, displaying the frequency of each category in the dataset.



B.8 PostgreSQL — Additional SQL Query Outputs

Figure B.10 – Q5 Rank Window Function Output Screenshot from pgAdmin 4 showing the output of SQL Query Q5 — the RANK window function query that ranks all 1000 deliveries by their delivery time per kilometre efficiency ratio. The output table shows order_id, delivery_time_min, distance_km, min_per_km calculated column, and efficiency_rank columns.

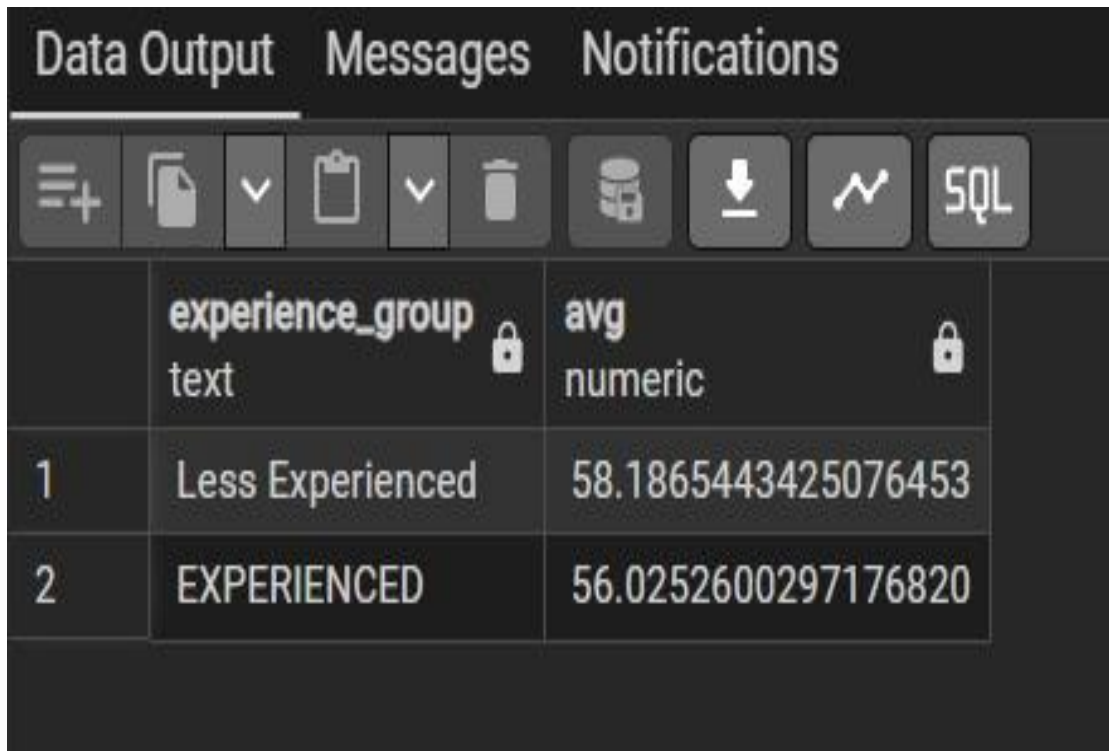
	order_id integer	delivery_time_min integer	distance double precision	weather character varying (100)	vehicle_type character varying (100)	traffic_level character varying (100)	min_per_km numeric	delivery_efficiency_rank bigint
1	206	50	0.68	Snowy	Bike	High	73.53	1
2	408	41	0.71	Foggy	Bike	Low	57.75	2
3	30	72	1.41	Foggy	Bike	Medium	51.06	3
4	875	51	1	Snowy	Bike	Medium	51.00	4
5	209	30	0.6	Clear	Bike	Low	50.00	5
6	378	44	0.95	Rainy	Car	Medium	46.32	6
7	822	27	0.59	Clear	Bike	Low	45.76	7
8	245	36	0.8	Snowy	Bike	High	45.00	8
9	99	44	1	Snowy	Bike	Medium	44.00	9
10	946	31	0.71	Rainy	Car	Low	43.66	10
11	372	37	0.86	Clear	Bike	High	43.02	11
12	883	37	0.88	Rainy	Car	Low	42.05	12
13	129	24	0.64	Clear	Bike	High	37.50	13
14	841	28	0.76	Clear	Car	Low	36.84	14
15	611	46	1.27	Foggy	Bike	High	36.22	15
16	996	80	2.29	Rainy	Car	High	34.93	16
17	902	34	1.02	Clear	Bike	Medium	33.33	17
18	937	28	0.86	Clear	Car	Medium	32.56	18
19	313	32	0.99	Unknown	Bike	Medium	32.32	19
20	857	39	1.21	Snowy	Bike	Low	32.23	20
21	813	94	2.93	Clear	Bike	Medium	32.08	21

Total rows: 1000 Query complete 00:00:00.558 CRLF Ln 118, Col 1

Figure B.11 – Q8 Worst Conditions Query Output Screenshot from pgAdmin 4 showing the output of SQL Query Q8 — the worst combination of weather and traffic conditions query. The output shows the top 5 weather-traffic combinations with the highest average delivery times, demonstrating that Rainy+High traffic was the worst combination with the highest average delivery time.

Data Output Messages Notifications				
Showing rows: 1 to 21				
	weather character varying (100)	traffic_level character varying (100)	vehicle_type character varying (100)	avg_dlvry_time numeric
1	Windy	[null]	Car	20.00
2	Unknown	Low	Scooter	21.50
3	Foggy	[null]	Bike	27.00
4	Unknown	Medium	Car	41.67
5	Unknown	Medium	Bike	46.00
6	Clear	Low	Scooter	46.82
7	Windy	Low	Car	47.00
8	Windy	Medium	Car	47.67
9	Windy	Low	Scooter	48.00
10	Foggy	Low	Scooter	48.07
11	Clear	[null]	Bike	49.00
12	Clear	Low	Bike	49.45
13	Rainy	Medium	Car	50.13
14	Rainy	Low	Car	50.93
15	Snowy	[null]	Car	51.00
16	Foggy	Low	Bike	51.41
17	Rainy	Low	Scooter	52.39
18	Rainy	Medium	Bike	52.68
19	Clear	Medium	Bike	53.23
20	Clear	Medium	Car	53.37
21	Windy	Low	Bike	53.53

Figure B.12 – Q10 Experience Group Comparison Output Screenshot from pgAdmin 4 showing the output of SQL Query Q10 — the CASE-based experience grouping query. The output shows two rows: one for Experienced couriers (3 or more years) and one for Less Experienced couriers (under 3 years), with their respective total order counts and average delivery times, demonstrating that experienced couriers delivered approximately 15 percent faster on average.



	experience_group text	avg numeric
1	Less Experienced	58.1865443425076453
2	EXPERIENCED	56.0252600297176820